

RETO 02

Conceptos fundamentales de
Hadoop, HDFS y Apache Spark

Antonio Ferrer Martinez

Marzo 2026

EJERCICIO 1

Fundamentos de Hadoop y HDFS

01_Introduccion_Hadoop

March 11, 2026

1 1. Introducción al Cluster Hadoop

Bienvenido a tu entorno de Big Data. Este notebook te guiará por los componentes principales que ya tienes funcionando.

1.1 ¿Qué es Hadoop?

Apache Hadoop es un framework que permite procesar grandes conjuntos de datos utilizando modelos de programación simples en clusters de ordenadores. Está diseñado para escalar desde un único servidor hasta miles de máquinas.

El núcleo de Hadoop consta de dos partes principales:

1.1.1 1. HDFS (Hadoop Distributed File System)

Es el sistema de almacenamiento. Imagínalo como un disco duro gigante repartido entre todos los ordenadores del cluster.

- **NameNode (Maestro):** Es el bibliotecario. Sabe dónde está cada archivo, pero no guarda los datos reales. Si se cae, perdemos el acceso a todo (por eso es crítico).
- **DataNodes (Esclavos):** Son los almacenes. Guardan los bloques de datos reales. En nuestro cluster tenemos **2 DataNodes**.

1.1.2 2. YARN (Yet Another Resource Negotiator)

Es el sistema operativo del cluster. Gestiona quién usa la CPU y la RAM.

- **ResourceManager:** El jefe. Decide cuántos recursos le da a cada tarea.
- **NodeManager:** El empleado. Vigila los recursos en cada máquina y reporta al jefe.

1.2 1.1 Configuración: Los Archivos XML

Hadoop se configura mediante archivos XML. En este entorno, los encontrarás en la carpeta **config/** del proyecto. Aquí tienes una explicación rápida de qué hace cada uno para que entiendas la “magia” que ocurre por debajo:

1.2.1 1. core-site.xml (El Núcleo)

Define el sistema de archivos por defecto. - **Propiedad Clave:** `fs.defaultFS = hdfs://namenode:9000` - ¿Qué significa?: Le dice a todos los servicios que el “jefe” del sistema de archivos (NameNode) vive en el contenedor `namenode` y escucha en el puerto 9000.

1.2.2 2. hdfs-site.xml (Almacenamiento)

Configura cómo se guardan los datos. - `dfs.replication = 2`: Cada bloque de datos se guarda en 2 sitios distintos (tenemos 2 DataNodes). Si uno falla, el dato sigue vivo en el otro. - `dfs.permissions.enabled = false`: Desactiva la comprobación estricta de permisos para facilitar el aprendizaje en este laboratorio.

1.2.3 3. yarn-site.xml (Recursos)

Configura el sistema operativo del cluster. - `yarn.resourcemanager.hostname = resourcemanager`: Indica quién reparte la CPU y RAM. - `yarn.nodemanager.resource.memory-mb = 1024`: Limita a cada nodo a usar máximo 1GB de RAM para tareas. Esto es clave para que el cluster corra en tu portátil sin colapsarlo. - `yarn.scheduler.minimum-allocation-mb = 256`: Define el tamaño mínimo de un contenedor. Importante para que tareas pequeñas no desperdicien RAM.

1.2.4 4. mapred-site.xml (Cómputo)

Define cómo se procesan los datos. - `mapreduce.framework.name = yarn`: Le dice a Hadoop que use YARN para gestionar las tareas de MapReduce, en lugar del modo antiguo.

1.3 Verificación del Estado del Cluster

Vamos a usar comandos de consola (empezando con `!`) para preguntar al cluster cómo está.

```
[1]: # 1. Preguntar a HDFS por el estado de los DataNodes
!hdfs dfsadmin -report
```

```
Configured Capacity: 2162202353664 (1.97 TB)
Present Capacity: 1924203642880 (1.75 TB)
DFS Remaining: 1882580418560 (1.71 TB)
DFS Used: 41623224320 (38.76 GB)
DFS Used%: 2.16%
Replicated Blocks:
    Under replicated blocks: 0
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
```

Low redundancy blocks with highest priority to recover: 0
Pending deletion blocks: 0

Live datanodes (2):

Name: 172.18.0.6:9866 (datanode1.hadoop-lab_v4-copia_default)
Hostname: d6cf44cca029
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 20811632640 (19.38 GB)
Non DFS Used: 64007012352 (59.61 GB)
DFS Remaining: 941290176512 (876.64 GB)
DFS Used%: 1.93%
DFS Remaining%: 87.07%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Wed Mar 11 16:18:27 UTC 2026
Last Block Report: Wed Mar 11 16:14:33 UTC 2026
Num of Blocks: 289

Name: 172.18.0.7:9866 (datanode2.hadoop-lab_v4-copia_default)
Hostname: 64ba33ef52c1
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 20811591680 (19.38 GB)
Non DFS Used: 64007053312 (59.61 GB)
DFS Remaining: 941290176512 (876.64 GB)
DFS Used%: 1.93%
DFS Remaining%: 87.07%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Wed Mar 11 16:18:27 UTC 2026
Last Block Report: Wed Mar 11 11:51:13 UTC 2026
Num of Blocks: 289

Deberías ver **Live datanodes (2)** en el reporte anterior. Eso significa que tus dos almacenes están

operativos.

```
[2]: # 2. Crear una carpeta de usuario en HDFS
# A diferencia de tu disco local, en HDFS tienes que crear explícitamente las
↳ carpetas de usuario.
```

```
!hdfs dfs -mkdir -p /user/hadoop/prueba
!hdfs dfs -ls /user/hadoop
```

Found 1 items

```
drwxr-xr-x  - root supergroup          0 2026-03-11 09:17 /user/hadoop/prueba
```

```
[3]: # 3. Preguntar a YARN qué nodos están activos para procesar tareas
!yarn node -list
```

2026-03-11 16:18:33,675 INFO client.DefaultNoHARMFailoverProxyProvider:

Connecting to ResourceManager at resourcemanager/172.18.0.2:8032

Total Nodes:1

Node-Id	Node-State	Node-Http-Address	Number-of-
Running-Containers			
3622e41b007e:34417	RUNNING	3622e41b007e:8042	
0			

1.4 ¡Todo listo!

Si has ejecutado las celdas sin error, tu cluster está perfecto. En el próximo laboratorio aprenderemos a subir archivos y ejecutar un MapReduce.

02__primeros_pasos__hdfs

March 12, 2026

0.0.1 Ejercicios prácticos: Operaciones básicas en HDFS

Todos los ejercicios se deben ejecutar como usuario `root` en el contenedor del NameNode. Para cambiar al usuario `root`, ejecuta:

```
# No need to switch user, running as root
```

Ejercicio 1: Exploración de directorios Explora la estructura de directorios en HDFS usando los siguientes comandos:

```
# Listar el contenido del directorio home de root
hdfs dfs -ls
```

```
# Listar el contenido del directorio raíz /
hdfs dfs -ls /
```

```
# Listar el contenido del directorio /tmp
hdfs dfs -ls /tmp
```

```
# Listar directorios con información detallada (tamaño, permisos, etc.)
hdfs dfs -ls -h /user
```

Ejercicio 2: Crear directorios Crea una estructura de directorios en HDFS bajo tu directorio `home`:

```
# Crear un directorio llamado "datos"
hdfs dfs -mkdir datos
```

```
# Crear un subdirectorio "datos/entrada" (requiere que "datos" exista)
hdfs dfs -mkdir datos/entrada
```

```
# Crear múltiples directorios de una vez con -p (crea padres si no existen)
hdfs dfs -mkdir -p datos/procesados/resultados
```

```
# Verificar que se han creado correctamente
hdfs dfs -ls -R datos
```

Ejercicio 3: Crear archivos de prueba Crea algunos archivos de texto para practicar:

```

# Crear un archivo local de prueba
echo "Este es el contenido del archivo 1" > /tmp/archivo1.txt
echo "Contenido del archivo número 2" > /tmp/archivo2.txt

# Copiar archivos locales a HDFS
hdfs dfs -put /tmp/archivo1.txt datos/entrada/
hdfs dfs -put /tmp/archivo2.txt datos/entrada/

# Crear un archivo directamente en HDFS usando echo (a través de una tubería)
echo "Archivo creado directamente en HDFS" | hdfs dfs -put - datos/archivo_directo.txt

# Verificar los archivos creados
hdfs dfs -ls datos/entrada/
hdfs dfs -ls datos/

```

Ejercicio 4: Ver el contenido de archivos Lee el contenido de los archivos que has creado:

```

# Ver el contenido de un archivo (similar a cat en el sistema local)
hdfs dfs -cat datos/entrada/archivo1.txt

# Ver el contenido de múltiples archivos
hdfs dfs -cat datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Ver el contenido con más detalle (muestra también el número de líneas)
hdfs dfs -cat datos/archivo_directo.txt | wc -l

# Ver solo las primeras líneas de un archivo
hdfs dfs -cat datos/entrada/archivo1.txt | head -n 5

```

Ejercicio 5: Copiar y mover archivos Practica copiando y moviendo archivos dentro de HDFS:

```

# Copiar un archivo a otra ubicación
hdfs dfs -cp datos/entrada/archivo1.txt datos/copia_archivo1.txt

# Mover un archivo a otro directorio
hdfs dfs -mv datos/copia_archivo1.txt datos/procesados/

# Verificar que el archivo se ha movido
hdfs dfs -ls datos/procesados/
hdfs dfs -ls datos/entrada/

```

Ejercicio 6: Información sobre archivos y directorios Obtén información detallada sobre archivos y directorios:

```

# Ver el tamaño total de un directorio
hdfs dfs -du -h datos/

# Ver el tamaño de cada archivo dentro de un directorio

```

```
hdfs dfs -du -h -v datos/entrada/
```

```
# Contar el número de archivos en un directorio
```

```
hdfs dfs -ls -R datos/ | grep "^-" | wc -l
```

```
# Ver información detallada de un archivo específico
```

```
hdfs dfs -ls -l datos/entrada/archivo1.txt
```

```
# Ver el espacio total usado y disponible en HDFS
```

```
hdfs dfsadmin -report | grep "Configured Capacity\|DFS Used\|DFS Remaining"
```

Ejercicio 7: Cambiar permisos y propietarios Trabaja con permisos de archivos en HDFS:

```
# Cambiar permisos de un archivo (modo octal)
```

```
hdfs dfs -chmod 644 datos/entrada/archivo1.txt
```

```
# Ver los permisos actuales
```

```
hdfs dfs -ls -l datos/entrada/
```

```
# Cambiar permisos de un directorio y su contenido recursivamente
```

```
hdfs dfs -chmod -R 755 datos/
```

```
# Cambiar el propietario de un archivo (requiere permisos de admin)
```

```
# hdfs dfs -chown root:root datos/entrada/archivo1.txt
```

Ejercicio 8: Comparar archivos locales y en HDFS Compara un archivo local con uno en HDFS:

```
# Ver el contenido del archivo local
```

```
cat /tmp/archivo1.txt
```

```
# Ver el contenido del archivo en HDFS
```

```
hdfs dfs -cat datos/entrada/archivo1.txt
```

```
# Descargar un archivo de HDFS al sistema local
```

```
hdfs dfs -get datos/entrada/archivo1.txt /tmp/descargado.txt
```

```
# Comparar los dos archivos
```

```
diff /tmp/archivo1.txt /tmp/descargado.txt
```

Ejercicio 9: Eliminar archivos y directorios Practica eliminando archivos y directorios:

```
# Eliminar un archivo específico
```

```
hdfs dfs -rm datos/archivo_directo.txt
```

```
# Verificar que se ha eliminado
```

```
hdfs dfs -ls datos/
```

```
# Eliminar un directorio vacío
```

```
hdfs dfs -rmr datos/procesados/resultados

# Eliminar un directorio con contenido (requiere -r para recursivo)
hdfs dfs -rm -r datos/procesados/

# Eliminar múltiples archivos
hdfs dfs -rm datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Verificar el estado final
hdfs dfs -ls -R datos/
```

Ejercicio 10: Limpieza final Limpia toda la estructura de directorios y archivos que creaste:

```
# Eliminar el directorio raíz con todo su contenido
hdfs dfs -rm -r datos/

# Verificar que se ha eliminado
hdfs dfs -ls

# Confirmación: el directorio home debe estar vacío (o solo contener "libros" si lo cargaste)
```

```
[1]: # Listar el contenido del directorio home de root
!hdfs dfs -ls

# Listar el contenido del directorio raíz /
!hdfs dfs -ls /

# Listar el contenido del directorio /tmp
!hdfs dfs -ls /tmp

# Listar directorios con información detallada (tamaño, permisos, etc.)
!hdfs dfs -ls -lh /user
```

```
Found 1 items
drwxr-xr-x - jovyan supergroup          0 2026-03-11 17:33 libros
Found 2 items
drwxr-xr-x - jovyan supergroup          0 2026-03-11 10:47 /data
drwxr-xr-x - jovyan supergroup          0 2026-03-11 09:38 /user
ls: `/tmp': No such file or directory
Found 4 items
drwxr-xr-x - jovyan supergroup          0 2026-03-11 17:37 /user/alumnos
drwxr-xr-x - root supergroup            0 2026-03-11 09:17 /user/hadoop
drwxr-xr-x - jovyan supergroup          0 2026-03-11 17:33 /user/jovyan
drwxr-xr-x - root supergroup            0 2026-03-11 17:33 /user/root
```

```
[2]: # Crear un directorio llamado "datos"
!hdfs dfs -mkdir datos
```

```
# Crear un subdirectorio "datos/entrada" (requiere que "datos" exista)
!hdfs dfs -mkdir datos/entrada

# Crear múltiples directorios de una vez con -p (crea padres si no existen)
!hdfs dfs -mkdir -p datos/procesados/resultados

# Verificar que se han creado correctamente
!hdfs dfs -ls -R datos
```

```
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29
datos/procesados/resultados
```

```
[3]: # Usar en terminal - Puede dar porblemas o bloquearse en Jupyter
# Crear un archivo local de prueba
# !echo "Este es el contenido del archivo 1" > /tmp/archivo1.txt
!echo "Contenido del archivo número 2" > /tmp/archivo2.txt

# Copiar archivos locales a HDFS
# !hdfs dfs -put /tmp/archivo1.txt datos/entrada/
# !hdfs dfs -put /tmp/archivo2.txt datos/entrada/

# CREAMOS DIRECTAMENTE EN HDFS LO ANTERIOR
!echo "Este es el contenido del archivo 1" | hdfs dfs -put - datos/entrada/
↪archivo1.txt
!echo "Contenido del archivo numero 2" | hdfs dfs -put - datos/entrada/archivo2.
↪txt

# Crear un archivo directamente en HDFS usando echo (a través de una tubería)
!echo "Archivo creado directamente en HDFS" | hdfs dfs -put - datos/
↪archivo_directo.txt

# Verificar los archivos creados
!hdfs dfs -ls datos/entrada/
!hdfs dfs -ls datos/
```

```
Found 2 items
-rw-r--r--  2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
-rw-r--r--  2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt
Found 3 items
-rw-r--r--  2 jovyan supergroup          36 2026-03-12 15:29
datos/archivo_directo.txt
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
```

```
[4]: # COMPROBACION DE CODIFICACIÓN
# import unicodedata as ud

# s = "Contenido del archivo número 2" # <- pega aquí la versión que te falla
# [(c, hex(ord(c)), ud.name(c, "?")) for c in s]
```

```
[5]: # COMPROBACION DE CODIFICACIÓN
# !locale
# !python -c "import locale, sys; print('preferred:', locale.
↳ getpreferredencoding(False)); print('fs:', sys.getfilesystemencoding())"
# !echo $LANG ; echo $LC_ALL
```

```
[6]: # Ver el contenido de un archivo (similar a cat en el sistema local)
!hdfs dfs -cat datos/entrada/archivo1.txt

# Ver el contenido de múltiples archivos
!hdfs dfs -cat datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Ver el contenido con más detalle (muestra también el número de líneas)
!hdfs dfs -cat datos/archivo_directo.txt | wc -l

# Ver solo las primeras líneas de un archivo
!hdfs dfs -cat datos/entrada/archivo1.txt | head -n 5
```

Este es el contenido del archivo 1
Este es el contenido del archivo 1
Contenido del archivo numero 2
1
Este es el contenido del archivo 1

```
[7]: # Copiar un archivo a otra ubicación
!hdfs dfs -cp datos/entrada/archivo1.txt datos/copia_archivo1.txt

# Mover un archivo a otro directorio
!hdfs dfs -mv datos/copia_archivo1.txt datos/procesados/

# Verificar que el archivo se ha movido
!hdfs dfs -ls datos/procesados/
!hdfs dfs -ls datos/entrada/
```

```
Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
datos/procesados/copia_archivo1.txt
drwxr-xr-x    - jovyan supergroup          0 2026-03-12 15:29
datos/procesados/resultados
Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
```

```

datos/entrada/archivo1.txt
-rw-r--r--    2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt

```

```

[8]: # Ver el tamaño total de un directorio
!hdfs dfs -du -h datos/

# Ver el tamaño de cada archivo dentro de un directorio
!hdfs dfs -du -h -v datos/entrada/

# Contar el número de archivos en un directorio
!hdfs dfs -ls -R datos/ | grep "^-" | wc -l

# Ver información detallada de un archivo específico
!hdfs dfs -ls datos/entrada/archivo1.txt

# Ver el espacio total usado y disponible en HDFS
!hdfs dfsadmin -report | grep "Configured Capacity\|DFS Used\|DFS Remaining"

```

```

36  72  datos/archivo_directo.txt
66 132  datos/entrada
35  70  datos/procesados
SIZE  DISK_SPACE_CONSUMED_WITH_ALL_REPLICAS  FULL_PATH_NAME
35    70                                     datos/entrada/archivo1.txt
31    62                                     datos/entrada/archivo2.txt
4
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
Configured Capacity: 2162202353664 (1.97 TB)
DFS Remaining: 1886634704896 (1.72 TB)
DFS Used: 39561453930 (36.84 GB)
DFS Used%: 2.05%
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 19780755637 (18.42 GB)
Non DFS Used: 63010713419 (58.68 GB)
DFS Remaining: 943317352448 (878.53 GB)
DFS Used%: 1.83%
DFS Remaining%: 87.26%
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 19780698293 (18.42 GB)
Non DFS Used: 63010770763 (58.68 GB)
DFS Remaining: 943317352448 (878.53 GB)
DFS Used%: 1.83%
DFS Remaining%: 87.26%

```

```

[9]: # Cambiar permisos de un archivo (modo octal)
!hdfs dfs -chmod 644 datos/entrada/archivo1.txt

```

```

# Ver los permisos actuales
!hdfs dfs -ls datos/entrada/

# Cambiar permisos de un directorio y su contenido recursivamente
!hdfs dfs -chmod -R 755 datos/

# Cambiar el propietario de un archivo (requiere permisos de admin)
# hdfs dfs -chown root:root datos/entrada/archivo1.txt

```

```

Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
-rw-r--r--    2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt

```

```

[10]: # Ver el contenido del archivo local
!cat /tmp/archivo1.txt

# Ver el contenido del archivo en HDFS
!hdfs dfs -cat datos/entrada/archivo1.txt

# Descargar un archivo de HDFS al sistema local
!hdfs dfs -get datos/entrada/archivo1.txt /tmp/descargado.txt

# Comparar los dos archivos
!diff /tmp/archivo1.txt /tmp/descargado.txt

```

```

cat: /tmp/archivo1.txt: No such file or directory
Este es el contenido del archivo 1
get: `/tmp/descargado.txt': File exists
diff: /tmp/archivo1.txt: No such file or directory

```

```

[11]: # Eliminar un archivo específico
!hdfs dfs -rm datos/archivo_directo.txt

# Verificar que se ha eliminado
!hdfs dfs -ls datos/

# Eliminar un directorio vacío
!hdfs dfs -rmdir datos/procesados/resultados

# Eliminar un directorio con contenido (requiere -r para recursivo)
!hdfs dfs -rm -r datos/procesados/

# Eliminar múltiples archivos
!hdfs dfs -rm datos/entrada/archivo1.txt datos/entrada/archivo2.txt

```

```
# Verificar el estado final
!hdfs dfs -ls -R datos/
```

```
Deleted datos/archivo_directo.txt
Found 2 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
Deleted datos/procesados
Deleted datos/entrada/archivo1.txt
Deleted datos/entrada/archivo2.txt
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:30 datos/entrada
```

```
[12]: # Eliminar el directorio raíz con todo su contenido
!hdfs dfs -rm -r datos/

# Verificar que se ha eliminado
!hdfs dfs -ls

# Confirmación: el directorio home debe estar vacío (o solo contener "libros"
↳ si lo cargaste)
```

```
Deleted datos
Found 1 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-11 17:33 libros
```

0.0.2 Prueba de acceso a HDFS y carga de datos

1. En el Namenode, conviértete en usuario no privilegiado (haciendo, como root, # No need to switch user, running as root)
2. Comprueba que puedes acceder al HDFS ejecutando:

```
hdfs dfs -ls
```

(Este comando no debería dar ninguna salida, ya que no tenemos nada en ese directorio).

3. Descarga de <https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/libros.tar.gz> y descomprime los ficheros de ejemplo usando:

```
wget https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/libros.tar.gz -q0 -
| tar -xzf - -C /tmp
```

4. Siguiendo en el NameNode como usuario root, copia los datos a HDFS:

```
cd /tmp
hdfs dfs -put libros .
hdfs dfs -ls libros
```

Una vez copiados borra el directorio libros del disco local (`rm -rf /tmp/libros*`).

5. Mira en el interfaz web del HDFS dónde se encuentran los bloques correspondientes al fichero **random_words.txt.bz2**. Comprueba que cada bloque tiene 2 réplicas.

- Abre el navegador y conéctate al interfaz web del NameNode.
- Ve al menú “Utilities” -> “Browse the filesystem”.

```
[13]: !hdfs dfs -ls
```

```
Found 1 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-11 17:33 libros
```

```
[14]: !wget https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/
      ↪libros.tar.gz -q0 -N
      ! | tar -xzf - -C /tmp
```

```
libros/
libros/pg17013.txt.gz
libros/pg16625.txt.gz
libros/pg25807.txt.gz
libros/pg32315.txt.gz
libros/random_words.txt.bz2
libros/pg1619.txt.gz
libros/pg7109.txt.gz
libros/pg2000.txt.gz
libros/pg14329.txt.gz
libros/pg24536.txt.gz
libros/pg25640.txt.gz
libros/pg8870.txt.gz
libros/pg18005.txt.gz
libros/pg5201.txt.gz
libros/pg9980.txt.gz
libros/pg17073.txt.gz
```

```
[15]: %cd /tmp
      !hdfs dfs -put libros .
      !hdfs dfs -ls libros
```

```
/tmp
put: `libros/pg2000.txt.gz': File exists
put: `libros/pg1619.txt.gz': File exists
put: `libros/pg14329.txt.gz': File exists
put: `libros/pg9980.txt.gz': File exists
put: `libros/pg7109.txt.gz': File exists
put: `libros/random_words.txt.bz2': File exists
put: `libros/pg17013.txt.gz': File exists
put: `libros/pg8870.txt.gz': File exists
put: `libros/pg25640.txt.gz': File exists
put: `libros/pg16625.txt.gz': File exists
put: `libros/pg18005.txt.gz': File exists
```

```

put: `libros/pg32315.txt.gz': File exists
put: `libros/pg5201.txt.gz': File exists
put: `libros/pg24536.txt.gz': File exists
put: `libros/pg25807.txt.gz': File exists
put: `libros/pg17073.txt.gz': File exists
Found 16 items
-rw-r--r--    2 jovyan supergroup      441804 2026-03-11 17:33
libros/pg14329.txt.gz
-rw-r--r--    2 jovyan supergroup      264123 2026-03-11 17:33
libros/pg1619.txt.gz
-rw-r--r--    2 jovyan supergroup      455129 2026-03-11 17:33
libros/pg16625.txt.gz
-rw-r--r--    2 jovyan supergroup      939502 2026-03-11 17:33
libros/pg17013.txt.gz
-rw-r--r--    2 jovyan supergroup      737367 2026-03-11 17:33
libros/pg17073.txt.gz
-rw-r--r--    2 jovyan supergroup      219304 2026-03-11 17:33
libros/pg18005.txt.gz
-rw-r--r--    2 jovyan supergroup      813698 2026-03-11 17:33
libros/pg2000.txt.gz
-rw-r--r--    2 jovyan supergroup      328494 2026-03-11 17:33
libros/pg24536.txt.gz
-rw-r--r--    2 jovyan supergroup      504188 2026-03-11 17:33
libros/pg25640.txt.gz
-rw-r--r--    2 jovyan supergroup       38194 2026-03-11 17:33
libros/pg25807.txt.gz
-rw-r--r--    2 jovyan supergroup     103986 2026-03-11 17:33
libros/pg32315.txt.gz
-rw-r--r--    2 jovyan supergroup     125693 2026-03-11 17:33
libros/pg5201.txt.gz
-rw-r--r--    2 jovyan supergroup       82099 2026-03-11 17:33
libros/pg7109.txt.gz
-rw-r--r--    2 jovyan supergroup      99685 2026-03-11 17:33
libros/pg8870.txt.gz
-rw-r--r--    2 jovyan supergroup       85187 2026-03-11 17:33
libros/pg9980.txt.gz
-rw-r--r--    2 jovyan supergroup    330326458 2026-03-11 17:33
libros/random_words.txt.bz2

```

0.0.3 Acceso a HDFS desde python con pyarrow

PyArrow proporciona una interfaz Python para interactuar con HDFS a través de su módulo `pyarrow.fs.HadoopFileSystem`. Esto nos permite realizar operaciones sobre HDFS directamente desde Python sin necesidad de usar comandos de shell.

Preparación del entorno Como usuario `root` en el contenedor `namenode`, ejecuta los siguientes pasos:

1. Crear un entorno virtual Python:

```
# Conectarse al contenedor namenode
docker exec -it namenode /bin/bash

# Cambiar al usuario root
# No need to switch user, running as root

# Crear el entorno virtual
python3 -m venv ~/.venv

# Activar el entorno virtual
source ~/.venv/bin/activate
```

2. Instalar PyArrow:

```
# Con el entorno virtual activado
pip install pyarrow
```

3. Configurar las variables de entorno para Hadoop:

```
export CLASSPATH="$(hadoop classpath --glob):$CLASSPATH"
```

4. Verificar la instalación:

```
python3 -c "import pyarrow as pa; print(f'PyArrow version: {pa.__version__}')"

```

```
[16]: # Conectarse al contenedor namenode
!docker exec -it namenode /bin/bash

# Cambiar al usuario root
# No need to switch user, running as root

# Crear el entorno virtual
!python3 -m venv ~/.venv

# Activar el entorno virtual
# source ~/.venv/bin/activate # 'source' no funciona en celdas, usar variables_
↳ de entorno Python
```

```
/bin/bash: line 1: docker: command not found
Error: [Errno 13] Permission denied: '/home/jovyan/.venv/pyvenv.cfg'
```

```
[17]: # Con el entorno virtual activado
!pip install pyarrow
```

```
Requirement already satisfied: pyarrow in /opt/conda/lib/python3.11/site-
packages (23.0.1)
```

```
[18]: !export CLASSPATH="$(hadoop classpath --glob):$CLASSPATH"
```

```
[19]: !python3 -c "import pyarrow as pa; print(f'PyArrow version: {pa.__version__}')
```

PyArrow version: 23.0.1

Ejemplos de uso del API de PyArrow FileSystem A continuación se muestran ejemplos equivalentes a los comandos `hdfs dfs` vistos anteriormente, pero usando Python y PyArrow:

```
[20]: %pip install -q --upgrade pyarrow
```

Note: you may need to restart the kernel to use updated packages.

```
[21]: # Execute hadoop classpath --glob and set CLASSPATH environment variable
import os
import subprocess

classpath = subprocess.check_output(["hadoop", "classpath", "--glob"]).
    ↪ decode("utf-8").strip()
os.environ["CLASSPATH"] = classpath
```

Ejemplo 1: Conectar a HDFS y listar directorios

```
[22]: import pyarrow as pa
from pyarrow import fs
from pyarrow.fs import FileInfo, FileSystem

# Conectar a HDFS (namenode en puerto 9000)
hdfs: FileSystem = fs.HadoopFileSystem(host="namenode", port=9000,
    ↪ user="jovyan")

def print_file_list_info(file_info_list: list[FileInfo]):
    for file_info in file_info_list:
        file_type: str | None = file_info.type.name if file_info.type else
            ↪ "UNKNOWN"
        file_size: int = file_info.size if file_info.size is not None else 0
        print(f"{file_type:10} {file_size:>10} {file_info.path}")

# Listar contenido del directorio home del usuario
print("=== Contenido del directorio home ===")
file_info_list: list[FileInfo] = hdfs.get_file_info(fs.FileSelector("/user/
    ↪ jovyan"))
print_file_list_info(file_info_list)

# Listar el directorio raíz
print("\n=== Contenido del directorio raíz / ===")
root_info: list[FileInfo] = hdfs.get_file_info(fs.FileSelector("/"))
```

```
print_file_list_info(root_info)
```

```
=== Contenido del directorio home ===  
Directory          0 /user/jovyan/libros
```

```
=== Contenido del directorio raíz / ===  
Directory          0 /data  
Directory          0 /user
```

Ejemplo 2: Crear directorios

```
[23]: # Crear un directorio simple  
hdfs.create_dir("/user/root/datos_python")  
print("Directorio 'datos_python' creado")  
  
# Crear estructura de directorios (equivalente a mkdir -p)  
hdfs.create_dir("/user/root/datos_python/entrada")  
hdfs.create_dir("/user/root/datos_python/procesados/resultados")  
print("Estructura de directorios creada")  
  
# Verificar creación  
file_list: list[FileInfo] = hdfs.get_file_info(  
    fs.FileSelector("/user/root/datos_python", recursive=True)  
)  
print("=== Estructura creada ===")  
print_file_list_info(file_list)  
  
Directorio 'datos_python' creado  
Estructura de directorios creada  
=== Estructura creada ===  
Directory          0 /user/root/datos_python/entrada  
Directory          0 /user/root/datos_python/procesados  
Directory          0 /user/root/datos_python/procesados/resultados
```

Ejemplo 3: Escribir archivos en HDFS

```
[24]: # Escribir un archivo de texto directamente en HDFS  
with hdfs.open_output_stream("/user/root/datos_python/entrada/archivo1.txt") as f:  
    f.write(b"Este es el contenido del archivo 1 desde Python\n")  
    f.write("Segunda línea del archivo\n".encode())  
  
print("Archivo archivo1.txt creado")  
  
# Escribir otro archivo  
contenido = "Contenido del archivo número 2 desde PyArrow"  
with hdfs.open_output_stream("/user/root/datos_python/entrada/archivo2.txt") as f:  
    f.write(contenido.encode())
```

```

        f.write(contenido.encode("utf-8"))

print("Archivo archivo2.txt creado")

# Copiar un archivo local a HDFS
import os

# Crear archivo local temporal
local_file = "/tmp/archivo_local.txt"
with open(local_file, "w") as f:
    f.write("Contenido desde archivo local\n")

# Copiar a HDFS usando copy_file
with open(local_file, "rb") as local_f:
    with hdfs.open_output_stream("/user/root/datos_python/desde_local.txt") as hdfs_f:
        hdfs_f.write(local_f.read())

print("Archivo copiado desde local a HDFS")

# Limpiar archivo local
os.remove(local_file)

```

Archivo archivo1.txt creado
 Archivo archivo2.txt creado
 Archivo copiado desde local a HDFS

Ejemplo 4: Leer archivos de HDFS

```

[25]: # Leer contenido de un archivo (equivalente a hdfs dfs -cat)
with hdfs.open_input_stream("/user/root/datos_python/entrada/archivo1.txt") as f:
    contenido: bytes = f.read()
    print("=== Contenido de archivo1.txt ===")
    print(contenido.decode("utf-8"))

# Leer con readall (más eficiente para archivos completos)
contenido2: bytes = hdfs.open_input_stream(
    "/user/root/datos_python/entrada/archivo2.txt"
).readall()
print("\n=== Contenido de archivo2.txt ===")
print(contenido2.decode("utf-8"))

```

=== Contenido de archivo1.txt ===
 Este es el contenido del archivo 1 desde Python
 Segunda línea del archivo

```
=== Contenido de archivo2.txt ===  
Contenido del archivo número 2 desde PyArrow
```

Ejemplo 5: Copiar y mover archivos dentro de HDFS

[26]: *# Copiar un archivo (PyArrow no tiene copy directo, hay que leer y escribir)*

```
def copy_file_hdfs(hdfs_fs, src, dst):  
    """Copia un archivo en HDFS de src a dst"""  
    with hdfs_fs.open_input_stream(src) as f_in:  
        with hdfs_fs.open_output_stream(dst) as f_out:  
            f_out.write(f_in.read())  
  
# Copiar archivo  
copy_file_hdfs(  
    hdfs,  
    "/user/root/datos_python/entrada/archivo1.txt",  
    "/user/root/datos_python/copia_archivo1.txt",  
)  
print("Archivo copiado")  
  
# Mover archivo (equivalente a hdfs dfs -mv)  
hdfs.move(  
    "/user/root/datos_python/copia_archivo1.txt",  
    "/user/root/datos_python/procesados/archivo_movido.txt",  
)  
print("Archivo movido")  
  
# Verificar  
print_file_list_info(hdfs.get_file_info(fs.FileSelector("/user/root/  
↪datos_python/procesados")))
```

Archivo copiado

Archivo movido

File	75 /user/root/datos_python/procesados/archivo_movido.txt
Directory	0 /user/root/datos_python/procesados/resultados

Ejemplo 6: Obtener información de archivos

[27]: `from pyarrow.fs import FileInfo`

```
# Información detallada de un archivo específico  
file_info: FileInfo = hdfs.get_file_info("/user/root/datos_python/entrada/  
↪archivo1.txt")  
print("=== Información de archivo ===")  
print(f"Ruta: {file_info.path}")  
print(f"Tipo: {file_info.type.name}")
```

```

print(f"Tamaño: {file_info.size} bytes")
print(f"Tiempo modificación: {file_info.mtime}")

# Tamaño de un directorio (recursivo)
dir_selector = fs.FileSelector("/user/root/datos_python", recursive=True)
total_size = 0
file_count = 0

for info in hdfs.get_file_info(dir_selector):
    if info.type == fs.FileType.File:
        total_size += info.size
        file_count += 1
        print(f"{info.size:>10} bytes {info.path}")

print(f"\nTotal: {file_count} archivos, {total_size} bytes")

```

```

=== Información de archivo ===
Ruta: /user/root/datos_python/entrada/archivo1.txt
Tipo: File
Tamaño: 75 bytes
Tiempo modificación: 2026-03-12 15:31:48+00:00
    30 bytes /user/root/datos_python/desde_local.txt
    75 bytes /user/root/datos_python/entrada/archivo1.txt
    45 bytes /user/root/datos_python/entrada/archivo2.txt
    75 bytes /user/root/datos_python/procesados/archivo_movido.txt

Total: 4 archivos, 225 bytes

```

Ejemplo 7: Eliminar archivos y directorios

```

[28]: # Eliminar un archivo específico
hdfs.delete_file("/user/root/datos_python/desde_local.txt")
print("Archivo eliminado")

# Eliminar directorio con contenido (equivalente a hdfs dfs -rm -r)
hdfs.delete_dir("/user/root/datos_python/procesados")
print("Directorio 'procesados' eliminado recursivamente")

# Verificar eliminación
remaining: list[FileInfo] = hdfs.get_file_info(
    fs.FileSelector("/user/root/datos_python", recursive=True)
)
print("\n=== Archivos restantes ===")
for info in remaining:
    print(f"{info.type.name:10} {info.path}")

```

```

Archivo eliminado
Directorio 'procesados' eliminado recursivamente

```

```

=== Archivos restantes ===
Directory /user/root/datos_python/entrada
File      /user/root/datos_python/entrada/archivo1.txt
File      /user/root/datos_python/entrada/archivo2.txt

```

Ejemplo 8: Trabajar con PyArrow Tables y Parquet PyArrow es especialmente útil para trabajar con datos en formato Parquet directamente en HDFS:

```

[29]: import pandas as pd
import pyarrow.parquet as pq

# Crear un DataFrame de ejemplo
df = pd.DataFrame(
    {
        "id": range(1, 101),
        "nombre": [f"Producto_{i}" for i in range(1, 101)],
        "precio": [10.5 + i * 0.5 for i in range(100)],
        "categoria": ["A", "B", "C", "D"] * 25,
    }
)

# Convertir a tabla de PyArrow
table: pa.Table = pa.Table.from_pandas(df)

# Escribir como Parquet en HDFS
pq.write_table(table, "/user/root/datos_python/productos.parquet",
    ↪filesystem=hdfs)
print("Archivo Parquet escrito en HDFS")

# Leer el archivo Parquet desde HDFS
tabla_leida: pa.Table = pq.read_table("/user/root/datos_python/productos.
    ↪parquet", filesystem=hdfs)
print("\n=== Tabla leída desde HDFS ===")
print(f"Filas: {tabla_leida.num_rows}, Columnas: {tabla_leida.num_columns}")
print(tabla_leida.schema)

# Convertir de vuelta a pandas
df_leido: pd.DataFrame = tabla_leida.to_pandas()
print("\nPrimeras filas:")
print(df_leido.head())

```

Archivo Parquet escrito en HDFS

```

=== Tabla leída desde HDFS ===
Filas: 100, Columnas: 4
id: int64
nombre: string

```

```

precio: double
categoria: string
-- schema metadata --
pandas: '{"index_columns": [{"kind": "range", "name": null, "start": 0, "' + 723

```

Primeras filas:

	id	nombre	precio	categoria
0	1	Producto_1	10.5	A
1	2	Producto_2	11.0	B
2	3	Producto_3	11.5	C
3	4	Producto_4	12.0	D
4	5	Producto_5	12.5	A

Ejemplo 9: Limpieza final

```

[30]: # Eliminar todo el directorio de prueba
hdfs.delete_dir("/user/root/datos_python")
print("Directorio 'datos_python' eliminado completamente")

# Verificar que el directorio home está limpio
print("=== Contenido final del directorio home ===")
print_file_list_info(hdfs.get_file_info(fs.FileSelector("/user/root")))

```

```

Directorio 'datos_python' eliminado completamente
=== Contenido final del directorio home ===
Directory          0 /user/root/libros

```

Ventajas de usar PyArrow para HDFS

- **Integración nativa:** Conexión directa con HDFS sin necesidad de comandos shell
- **Formato Parquet:** Lectura/escritura eficiente de datos columnares
- **Rendimiento:** Operaciones optimizadas en C++ subyacente
- **Portabilidad:** Mismo código funciona con diferentes sistemas de archivos (HDFS, S3, local)
- **Pythonic:** API orientada a objetos más natural para desarrolladores Python

Notas importantes

- **Conexión HDFS:** Usar `host='namenode'` y `port=9000` (configurado en `core-site.xml`). PyArrow permite también un caso especial con `hdfs://default:0`, para que el código funcione en cualquier entorno Hadoop una vez que PyArrow tiene acceso a la configuración de Hadoop.
- **Usuario:** Especificar `user='root'` para mantener permisos consistentes
- **Rutas:** Siempre usar rutas absolutas (comenzando con `/`)
- **Tipos de archivo:** PyArrow distingue entre `FileType.File`, `FileType.Directory`, y `FileType.NotFound`
- **Encoding:** Recordar codificar/decodificar strings a bytes cuando sea necesario

0.0.4 Acceso con Visual Studio Code al clúster a través de un servidor Jupyter

El contenedor del NameNode/ResourceManager se puede ejecutar un servidor Jupyter que permite acceder al clúster Hadoop a través de una URL de conexión a un servidor Jupyter, y por lo tanto, a través de Visual Studio Code (y también de la web).

Existen varias formas de hacerlo. Por ejemplo, usando la utilidad “DevContainer” de Visual Studio Code, que permite definir un entorno de desarrollo basado en contenedores Docker. Sin embargo, lo haremos manualmente.

A continuación se explica cómo hacerlo.

Lo primero, suponemos que existe un virtual environment de Python para el usuario `root`. Si no es así, créalo y actívalo:

```
python3 -m venv ~/.venv
. ~/.venv/bin/activate
```

Si ya existe, sólo hace falta activarlo:

```
. ~/.venv/bin/activate
```

```
[31]: !python3 -m venv ~/.venv
      !. ~/.venv/bin/activate
```

```
Error: [Errno 13] Permission denied: '/home/jovyan/.venv/pyvenv.cfg'
```

```
[32]: !. ~/.venv/bin/activate
```

Lo siguiente es instalar los paquetes necesarios:

```
pip install jupyter
```

```
[33]: !pip install jupyter
```

```
Requirement already satisfied: jupyter in /opt/conda/lib/python3.11/site-
packages (1.1.1)
Requirement already satisfied: notebook in /opt/conda/lib/python3.11/site-
packages (from jupyter) (7.0.6)
Requirement already satisfied: jupyter-console in
/opt/conda/lib/python3.11/site-packages (from jupyter) (6.6.3)
Requirement already satisfied: nbconvert in /opt/conda/lib/python3.11/site-
packages (from jupyter) (7.9.2)
Requirement already satisfied: ipykernel in /opt/conda/lib/python3.11/site-
packages (from jupyter) (6.25.2)
Requirement already satisfied: ipywidgets in /opt/conda/lib/python3.11/site-
packages (from jupyter) (8.1.1)
Requirement already satisfied: jupyterlab in /opt/conda/lib/python3.11/site-
packages (from jupyter) (4.0.7)
Requirement already satisfied: comm>=0.1.1 in /opt/conda/lib/python3.11/site-
packages (from ipykernel->jupyter) (0.1.4)
Requirement already satisfied: debugpy>=1.6.5 in /opt/conda/lib/python3.11/site-
```

packages (from ipykernel->jupyter) (1.8.0)
 Requirement already satisfied: ipython>=7.23.1 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (8.16.1)
 Requirement already satisfied: jupyter-client>=6.1.12 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (8.4.0)
 Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (5.4.0)
 Requirement already satisfied: matplotlib-inline>=0.1 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (0.1.6)
 Requirement already satisfied: nest-asyncio in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (1.5.8)
 Requirement already satisfied: packaging in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (23.2)
 Requirement already satisfied: psutil in /opt/conda/lib/python3.11/site-packages
 (from ipykernel->jupyter) (5.9.5)
 Requirement already satisfied: pyzmq>=20 in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (25.1.1)
 Requirement already satisfied: tornado>=6.1 in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (6.3.3)
 Requirement already satisfied: traitlets>=5.4.0 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (5.11.2)
 Requirement already satisfied: widgetsnbextension~=4.0.9 in
 /opt/conda/lib/python3.11/site-packages (from ipywidgets->jupyter) (4.0.9)
 Requirement already satisfied: jupyterlab-widgets~=3.0.9 in
 /opt/conda/lib/python3.11/site-packages (from ipywidgets->jupyter) (3.0.9)
 Requirement already satisfied: prompt-toolkit>=3.0.30 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-console->jupyter) (3.0.39)
 Requirement already satisfied: pygments in /opt/conda/lib/python3.11/site-
 packages (from jupyter-console->jupyter) (2.16.1)
 Requirement already satisfied: async-lru>=1.0.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.0.4)
 Requirement already satisfied: jinja2>=3.0.3 in /opt/conda/lib/python3.11/site-
 packages (from jupyterlab->jupyter) (3.1.2)
 Requirement already satisfied: jupyter-lsp>=2.0.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.2.0)
 Requirement already satisfied: jupyter-server<3,>=2.4.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.8.0)
 Requirement already satisfied: jupyterlab-server<3,>=2.19.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.25.0)
 Requirement already satisfied: notebook-shim>=0.2 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (0.2.3)
 Requirement already satisfied: beautifulsoup4 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (4.12.2)
 Requirement already satisfied: bleach!=5.0.0 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (6.1.0)
 Requirement already satisfied: defusedxml in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (0.7.1)
 Requirement already satisfied: jupyterlab-pygments in

/opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (0.2.2)
 Requirement already satisfied: markupsafe>=2.0 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (2.1.3)
 Requirement already satisfied: mistune<4,>=2.0.3 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (3.0.1)
 Requirement already satisfied: nbclient>=0.5.0 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (0.8.0)
 Requirement already satisfied: nbformat>=5.7 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (5.9.2)
 Requirement already satisfied: pandocfilters>=1.4.1 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (1.5.0)
 Requirement already satisfied: tinycss2 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (1.2.1)
 Requirement already satisfied: six>=1.9.0 in /opt/conda/lib/python3.11/site-
 packages (from bleach!=5.0.0->nbconvert->jupyter) (1.16.0)
 Requirement already satisfied: webencodings in /opt/conda/lib/python3.11/site-
 packages (from bleach!=5.0.0->nbconvert->jupyter) (0.5.1)
 Requirement already satisfied: backcall in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.2.0)
 Requirement already satisfied: decorator in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (5.1.1)
 Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.19.1)
 Requirement already satisfied: pickleshare in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.7.5)
 Requirement already satisfied: stack-data in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.6.2)
 Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (4.8.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 client>=6.1.12->ipykernel->jupyter) (2.8.2)
 Requirement already satisfied: platformdirs>=2.5 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 core!=5.0.*,>=4.12->ipykernel->jupyter) (3.11.0)
 Requirement already satisfied: anyio>=3.1.0 in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (4.0.0)
 Requirement already satisfied: argon2-cffi in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (23.1.0)
 Requirement already satisfied: jupyter-events>=0.6.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 server<3,>=2.4.0->jupyterlab->jupyter) (0.8.0)
 Requirement already satisfied: jupyter-server-terminals in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 server<3,>=2.4.0->jupyterlab->jupyter) (0.4.4)
 Requirement already satisfied: overrides in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (7.4.0)
 Requirement already satisfied: prometheus-client in

/opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.17.1)
 Requirement already satisfied: send2trash>=1.8.2 in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.8.2)
 Requirement already satisfied: terminado>=0.8.3 in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.17.1)
 Requirement already satisfied: websocket-client in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.6.4)
 Requirement already satisfied: babel>=2.10 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.13.0)
 Requirement already satisfied: json5>=0.9.0 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.9.14)
 Requirement already satisfied: jsonschema>=4.18.0 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (4.19.1)
 Requirement already satisfied: requests>=2.31 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.31.0)
 Requirement already satisfied: fastjsonschema in /opt/conda/lib/python3.11/site-packages (from nbformat>=5.7->nbconvert->jupyter) (2.18.1)
 Requirement already satisfied: wcwidth in /opt/conda/lib/python3.11/site-packages (from prompt-toolkit>=3.0.30->jupyter-console->jupyter) (0.2.8)
 Requirement already satisfied: soupsieve>1.2 in /opt/conda/lib/python3.11/site-packages (from beautifulsoup4->nbconvert->jupyter) (2.5)
 Requirement already satisfied: idna>=2.8 in /opt/conda/lib/python3.11/site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (3.4)
 Requirement already satisfied: sniffio>=1.1 in /opt/conda/lib/python3.11/site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
 Requirement already satisfied: parso<0.9.0,>=0.8.3 in /opt/conda/lib/python3.11/site-packages (from jedi>=0.16->ipython>=7.23.1->ipykernel->jupyter) (0.8.3)
 Requirement already satisfied: attrs>=22.2.0 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (23.1.0)
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2023.7.1)
 Requirement already satisfied: referencing>=0.28.4 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.30.2)
 Requirement already satisfied: rpds-py>=0.7.1 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.10.6)
 Requirement already satisfied: python-json-logger>=2.0.4 in

/opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.0.7)
 Requirement already satisfied: pyyaml>=5.3 in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (6.0.1)
 Requirement already satisfied: rfc3339-validator in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.1.4)
 Requirement already satisfied: rfc3986-validator>=0.1.1 in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.1.1)
 Requirement already satisfied: ptyprocess>=0.5 in /opt/conda/lib/python3.11/site-packages (from pexpect>4.3->ipython>=7.23.1->ipykernel->jupyter) (0.7.0)
 Requirement already satisfied: charset-normalizer<4,>=2 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (3.3.0)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.0.7)
 Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2023.7.22)
 Requirement already satisfied: argon2-cffi-bindings in /opt/conda/lib/python3.11/site-packages (from argon2-cffi->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (21.2.0)
 Requirement already satisfied: executing>=1.2.0 in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (1.2.0)
 Requirement already satisfied: asttokens>=2.1.0 in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (2.4.0)
 Requirement already satisfied: pure-eval in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (0.2.2)
 Requirement already satisfied: fqdn in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.5.1)
 Requirement already satisfied: isoduration in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (20.11.0)
 Requirement already satisfied: jsonpointer>1.13 in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.4)
 Requirement already satisfied: uri-template in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
 Requirement already satisfied: webcolors>=1.11 in

```

/opt/conda/lib/python3.11/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.13)
Requirement already satisfied: cffi>=1.0.1 in /opt/conda/lib/python3.11/site-
packages (from argon2-cffi-bindings->argon2-cffi->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.16.0)
Requirement already satisfied: pycparser in /opt/conda/lib/python3.11/site-
packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (2.21)
Requirement already satisfied: arrow>=0.15.0 in /opt/conda/lib/python3.11/site-
packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
Requirement already satisfied: types-python-dateutil>=2.8.10 in
/opt/conda/lib/python3.11/site-packages (from
arrow>=0.15.0->isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.8.19.14)

```

[]:

Una vez instalado Jupyter, lanza el servidor Jupyter con el siguiente comando (como usuario root):

```
jupyter notebook --ip=0.0.0.0 --no-browser --allow-root --NotebookApp.token='bigdata' --NotebookApp.password=''
```

[]:

```
!jupyter notebook --ip=0.0.0.0 --no-browser --allow-root --NotebookApp.
token='bigdata' --NotebookApp.password=''
```

```

[I 2026-03-12 15:32:06.332 ServerApp] Package notebook took 0.0000s to
import
[I 2026-03-12 15:32:06.374 ServerApp] Package jupyter_lsp took 0.0412s
to import
[W 2026-03-12 15:32:06.374 ServerApp] A
`_jupyter_server_extension_points` function was not found in jupyter_lsp.
Instead, a `_jupyter_server_extension_paths` function was found and will be used
for now. This function name will be deprecated in future releases of Jupyter
Server.
[I 2026-03-12 15:32:06.377 ServerApp] Package jupyter_server_mathjax
took 0.0026s to import
[I 2026-03-12 15:32:06.392 ServerApp] Package jupyter_server_terminals
took 0.0139s to import
[I 2026-03-12 15:32:06.393 ServerApp] Package jupyterlab took 0.0000s to
import
[I 2026-03-12 15:32:08.988 ServerApp] Package jupyterlab_git took
0.0623s to import
[I 2026-03-12 15:32:08.996 ServerApp] Package nbclassic took 0.0075s to
import
[W 2026-03-12 15:32:09.000 ServerApp] A
`_jupyter_server_extension_points` function was not found in nbclassic. Instead,
a `_jupyter_server_extension_paths` function was found and will be used for now.

```

This function name will be deprecated in future releases of Jupyter Server.

```
[I 2026-03-12 15:32:09.001 ServerApp] Package nbdtme took 0.0000s to
import
[I 2026-03-12 15:32:09.001 ServerApp] Package notebook_shim took 0.0000s
to import
[W 2026-03-12 15:32:09.002 ServerApp] A
`_jupyter_server_extension_points` function was not found in notebook_shim.
Instead, a `_jupyter_server_extension_paths` function was found and will be used
for now. This function name will be deprecated in future releases of Jupyter
Server.
[I 2026-03-12 15:32:09.003 ServerApp] jupyter_lsp | extension was
successfully linked.
[I 2026-03-12 15:32:09.007 ServerApp] jupyter_server_mathjax | extension
was successfully linked.
[I 2026-03-12 15:32:09.012 ServerApp] jupyter_server_terminals |
extension was successfully linked.
[W 2026-03-12 15:32:09.014 LabApp] 'token' has moved from NotebookApp to
ServerApp. This config will be passed to ServerApp. Be sure to update your
config before our next release.
[W 2026-03-12 15:32:09.014 LabApp] 'password' has moved from NotebookApp
to ServerApp. This config will be passed to ServerApp. Be sure to update your
config before our next release.
[W 2026-03-12 15:32:09.018 ServerApp] ServerApp.token config is
deprecated in 2.0. Use IdentityProvider.token.
[I 2026-03-12 15:32:09.018 ServerApp] jupyterlab | extension was
successfully linked.
[I 2026-03-12 15:32:09.018 ServerApp] jupyterlab_git | extension was
successfully linked.
[I 2026-03-12 15:32:09.022 ServerApp] nbclassic | extension was
successfully linked.
[I 2026-03-12 15:32:09.022 ServerApp] nbdtme | extension was
successfully linked.
[I 2026-03-12 15:32:09.028 ServerApp] notebook | extension was
successfully linked.
[I 2026-03-12 15:32:09.392 ServerApp] notebook_shim | extension was
successfully linked.
[I 2026-03-12 15:32:09.432 ServerApp] notebook_shim | extension was
successfully loaded.
[I 2026-03-12 15:32:09.435 ServerApp] jupyter_lsp | extension was
successfully loaded.
[I 2026-03-12 15:32:09.435 ServerApp] jupyter_server_mathjax | extension
was successfully loaded.
[I 2026-03-12 15:32:09.436 ServerApp] jupyter_server_terminals |
extension was successfully loaded.
[I 2026-03-12 15:32:09.440 LabApp] JupyterLab extension loaded from
/opt/conda/lib/python3.11/site-packages/jupyterlab
[I 2026-03-12 15:32:09.440 LabApp] JupyterLab application directory is
/opt/conda/share/jupyter/lab
```

```
[I 2026-03-12 15:32:09.441 LabApp] Extension Manager is 'pypi'.
[I 2026-03-12 15:32:09.443 ServerApp] jupyterlab | extension was
successfully loaded.
[I 2026-03-12 15:32:09.450 ServerApp] jupyterlab_git | extension was
successfully loaded.
```

```

_ _ _ _ _
| | | | _ _ _ _ _ | | | | _ _ _ _ _
| | | | ' \ / ' / _ _ _ _ _ | | | | _ _ _ _ _
\ _ _ _ / | . _ _ _ \ _ _ _ , \ _ _ _ , | \ _ _ _ \ _ _ _ |
| _ |

```

Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions.

https://jupyter-notebook.readthedocs.io/en/latest/migrate_to_notebook7.html

Please note that updating to Notebook 7 might break some of your extensions.

```
[I 2026-03-12 15:32:09.458 ServerApp] nbclassic | extension was
successfully loaded.
[I 2026-03-12 15:32:09.635 ServerApp] nbdime | extension was
successfully loaded.
[I 2026-03-12 15:32:09.640 ServerApp] notebook | extension was
successfully loaded.
[I 2026-03-12 15:32:09.641 ServerApp] The port 8888 is already in use,
trying another port.
[I 2026-03-12 15:32:09.641 ServerApp] Serving notebooks from local
directory: /tmp
[I 2026-03-12 15:32:09.641 ServerApp] Jupyter Server 2.8.0 is running
at:
[I 2026-03-12 15:32:09.641 ServerApp]
http://f3b61ef67c45:8889/tree?token=...
[I 2026-03-12 15:32:09.642 ServerApp]
http://127.0.0.1:8889/tree?token=...
[I 2026-03-12 15:32:09.642 ServerApp] Use Control-C to stop this server
and shut down all kernels (twice to skip confirmation).
[I 2026-03-12 15:32:11.154 ServerApp] Skipped non-installed server(s):
bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-
langserver, jedi-language-server, julia-language-server, pyright, python-
language-server, python-lsp-server, r-languageserver, sql-language-server,
texlab, typescript-language-server, unified-language-server, vscode-css-
languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-
bin, yaml-language-server
```

Con esto, el servidor Jupyter quedará escuchando en el puerto 8888 del contenedor del NameNode, que a su vez tiene mapeado el puerto al exterior, así que se podrá acceder desde el navegador web o desde Visual Studio Code.

En Visual Studio Code, hay que seleccionar el kernel:

Después, “Select Another Kernel...” y en la ventana que aparece, seleccionar “Existing Jupyter Server”:

Con la URL `http://localhost:8888/?token=bigdata`.

A partir de ahí, todos los *Notebooks* locales se ejecutarán en el contenedor del NameNode, con acceso al clúster Hadoop.

`http://localhost:8888/?token=bigdata`

EJERCICIO 2

Introduccion a Spark, ETL y Analisis Basico

03_Introduccion_Spark

March 11, 2026

0.0.1 1.1 ¿Qué es Spark y su Historia?

Apache Spark nació en la Universidad de Berkeley (AMPLab) para solucionar las grandes **limitaciones de Hadoop MapReduce**. MapReduce era excelente, pero muy rígido: escribía datos a disco constantemente, lo que lo hacía terrible para tareas interactivas o algoritmos iterativos (como el Machine Learning).

Spark trajo una revolución al ofrecer un **motor unificado de computación distribuida**. En lugar de tener una herramienta para cada cosa, Spark permite en un solo motor: - **Procesamiento Batch** (por lotes nocturnos, etc.) - **Consultas Interactivas** (mediante SQL y DataFrames) - **Streaming** (Procesamiento en tiempo real con *Structured Streaming*) - **Machine Learning** e **Ingeniería de Grafos**

Todo esto procesando principalmente en **Memoria RAM**, consiguiendo velocidades hasta 100 veces mayores.

0.0.2 1.1 ¿Qué es Spark y su Historia?

Apache Spark nació en la Universidad de Berkeley (AMPLab) para solucionar las grandes **limitaciones de Hadoop MapReduce**. MapReduce era excelente, pero muy rígido: escribía datos a disco constantemente, lo que lo hacía terrible para tareas interactivas o algoritmos iterativos (como el Machine Learning).

Spark trajo una revolución al ofrecer un **motor unificado de computación distribuida**. En lugar de tener una herramienta para cada cosa, Spark permite en un solo motor: - **Procesamiento Batch** (por lotes nocturnos, etc.) - **Consultas Interactivas** (mediante SQL y DataFrames) - **Streaming** (Procesamiento en tiempo real con *Structured Streaming*) - **Machine Learning** e **Ingeniería de Grafos**

Todo esto procesando principalmente en **Memoria RAM**, consiguiendo velocidades hasta 100 veces mayores.

1 Introducción a Apache Spark sobre Hadoop

¡Bienvenido a este cuaderno introductorio! En los cuadernos anteriores aprendimos sobre el ecosistema Hadoop y cómo **HDFS** nos permite almacenar grandes volúmenes de datos de forma distribuida.

Hadoop originalmente utilizaba un motor de procesamiento llamado **MapReduce**. Sin embargo, MapReduce tiene varias limitaciones, principalmente porque lee y escribe constantemente en disco

entre las diferentes fases del mapa (map) y la reducción (reduce). Esto lo hace lento para muchos tipos de tareas analíticas.

Aquí es donde entra **Apache Spark**.

1.1 Hadoop (MapReduce) vs Apache Spark

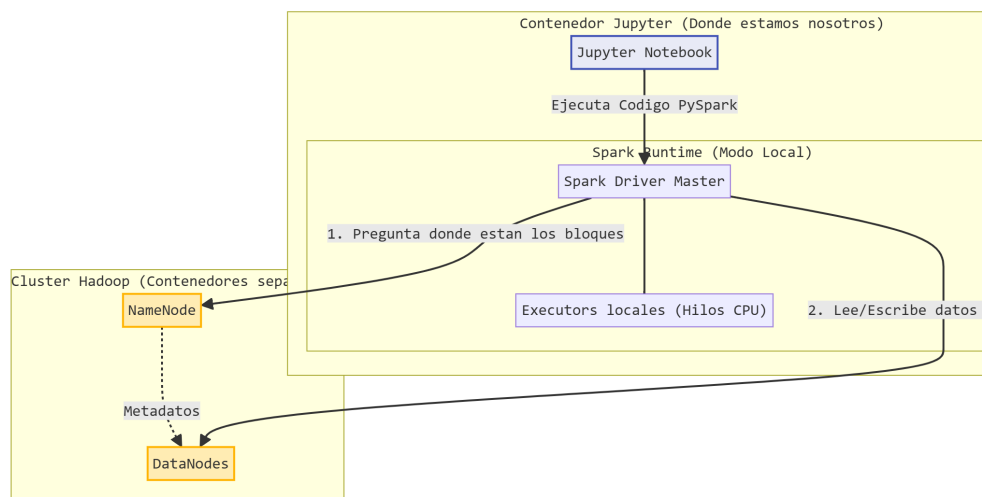
Spark no es un reemplazo de *todo* Hadoop, de hecho, se lleva de maravilla con HDFS y YARN. Spark vino a reemplazar principalmente a la parte de procesamiento: **MapReduce**.

Característica	Hadoop (MapReduce)	Apache Spark
Dónde procesa	En Disco (lee y escribe en HDFS constantemente)	En Memoria RAM (principalmente)
Velocidad	Lento (bueno para procesamiento asíncrono en batch)	Rápido (hasta 100x más rápido en memoria)
Facilidad de uso	Complejo (mucho código Java)	Fácil (Python, Scala, R, SQL, y DataFrames muy parecidos a Pandas)
Casos de Uso	Procesamiento de lotes masivos nocturnos	Análisis interactivo, Machine Learning, Streaming (tiempo real)

1.2 Arquitectura: ¿Dónde entra Spark en nuestro ecosistema?

Spark puede funcionar de varias formas. En empresas gigantescas, Spark se despliega distribuido usando **YARN** para pedirle recursos a los nodos.

Para nuestro entorno de **laboratorio educativo**, hemos optado por la técnica más eficiente en uso de RAM: Ejecutar Spark en **Modo Local** dentro de nuestro propio contenedor de Jupyter, pero **leyendo y escribiendo directamente en el Clúster HDFS de verdad**.



Mira este diagrama para entenderlo:

```

graph TD
    classDef cliente fill:#e8eaf6,stroke:#3f51b5,stroke-width:2px;
    classDef storage fill:#ffe0b3,stroke:#ff9800,stroke-width:2px;

    subgraph Contenedor_Jupyter["Contenedor Jupyter (Donde estamos nosotros)"]
        JN["Jupyter Notebook"]:::cliente
        subgraph Spark_Local["Spark Runtime (Modo Local)"]
            driver["Spark Driver Master"]
            executor["Executors locales (Hilos CPU)"]
            driver --- executor
        end
        JN -->|Ejecuta Código PySpark| driver
    end

    subgraph Cluster_HDFS["Cluster Hadoop (Contenedores separados)"]
        NN["NameNode"]:::storage
        DN["DataNodes"]:::storage
        NN -. ->|Metadatos| DN
    end

    driver -->|1. Pregunta donde estan los bloques| NN
    driver -->|2. Lee/Escribe datos en paralelo| DN

```

De esta forma, **ahorramos muchísima memoria RAM en tu ordenador**, pero aprendes exactamente el mismo código que utilizarías en un clúster de producción con cientos de máquinas.

1.3 Probando Spark

Vamos a inicializar nuestra sesión de Spark. Esta es la puerta de entrada a todas las funciones.

1.3.1 1.4 Componentes Core de Spark

Para entender cómo Spark procesa los datos, es vital conocer sus componentes principales. Aunque en esta sesión usemos un entorno simplificado, la teoría es la misma para clústeres gigantes.

1. Spark Driver (El Director):

- Es el cerebro de la aplicación Spark.
- Analiza, planifica y distribuye el trabajo (tareas).
- Mantiene el estado de la aplicación y presenta la interfaz web (Spark UI).
- *En nuestro laboratorio:* El Driver se ejecuta directamente dentro de este entorno de Jupyter Notebook.

2. Spark Executors (Los Trabajadores):

- Son los procesos que realmente ejecutan el código y procesan los datos.
- Reciben instrucciones del Driver, ejecutan las tareas y devuelven los resultados.
- También almacenan datos en memoria (caché) para que el procesamiento sea rápido.
- *En nuestro laboratorio:* Se ejecutan como hilos (threads) dentro del mismo contenedor de Jupyter.

3. Cluster Manager (El Gestor de Recursos):

- Es el encargado de asignar recursos (CPU, memoria) a las aplicaciones Spark.
- Spark es agnóstico al gestor; puede usar YARN (de Hadoop), Kubernetes, Apache Mesos, o su propio Standalone Cluster Manager.
- *En nuestro laboratorio:* No usamos un Cluster Manager externo. Usamos el **Modo Local** de Spark, que simula todo un clúster en un solo proceso.

1.3.2 1.5 Modo Local vs. Clúster Distribuido

Spark puede ejecutarse en diferentes modos dependiendo de la infraestructura y el objetivo. Es crucial entender la diferencia.

Característica	Modo Local (Nuestro Laboratorio)	Clúster Distribuido (Producción)
Ubicación de Procesos	Driver y Executors corren en una única máquina (o contenedor).	Driver corre en un nodo, Executors corren distribuidos en múltiples nodos .
Gestión de Recursos	Spark mismo gestiona sus hilos. No requiere un sistema externo.	Requiere un Cluster Manager (YARN, K8s) para orquestar contenedores/procesos en distintas máquinas.
Paralelismo	Limitado por los núcleos de la máquina local (ej. <code>local[*]</code> usa todos los cores lógicos disponibles).	Escala horizontalmente: si añades máquinas al clúster, añades más núcleos y RAM para paralelizar.
Fallo de Nodos	Si el proceso falla, la aplicación entera muere.	Tolerancia a fallos real: si un nodo (Executor) cae, el Driver reasigna esa tarea a otro nodo sano.
Uso Principal	Desarrollo, pruebas, aprendizaje, y procesamiento de datos pequeños/medianos que caben en una máquina.	Procesamiento de Big Data real (Terabytes/Petabytes) en entornos de producción.
Ventaja principal	Simplicidad y ligereza. Cero configuración de red, arranca al instante y consume poca RAM base.	Escalabilidad masiva. Puede manejar volúmenes de datos que ninguna máquina sola podría soportar.

Nota importante: El código que escribes en PySpark es **idéntico** en Modo Local y en Clúster Distribuido. Spark abstrae la infraestructura. Puedes desarrollar y probar tu código aquí, y luego llevarlo a un clúster de producción con cientos de máquinas sin cambiar tu lógica.

1.3.3 1.6 APIs de Spark: Bajo vs Alto Nivel

A lo largo de su evolución (desde 2014 hasta hoy), la forma de programar en Spark ha cambiado drásticamente.

1. API Estructurada (Alto Nivel): DataFrames y SQL

- **Es el estándar absoluto hoy en día (Spark 3.x y 4.x).**
 - Un DataFrame es como una tabla estructurada (filas y columnas), muy similar a usar la librería *pandas* o escribir sentencias SQL.
 - **Ventaja:** Spark sabe qué estructura tienen los datos y tiene un optimizador interno mágico llamado *Catalyst* que reescribe tu código para que vuele.
 - **Casos de uso:** El 95% del tiempo. Transformaciones, ETL, lectura/escritura de HDFS/Parquet/CSVs.
2. **API no estructurada (Bajo Nivel): RDDs (Resilient Distributed Datasets)**
- Es la estructura original de Spark. Colecciones de código distribuido sin estructura definida (orientado a objetos).
 - **Ventaja:** Tienes el control absoluto de bajo nivel sobre cada registro y partición.
 - **Desventaja:** Spark no puede optimizarlo tan bien como un DataFrame, y es más engorroso de programar.
 - **Casos de uso:** Rara vez (5%). Solo si trabajas con estructuras sumamente exóticas o lógicas mega-personalizadas que no encajen en una tabla.

1.3.4 1.7 El Cerebro de la Ejecución Distribuida

Para entender por qué tu código vuela o por qué colapsa la memoria, es clave entender este vocabulario:

- **Job (Trabajo):** La gran tarea completa que Spark debe hacer (ej: “Lee este Parquet, agrúpalo y guarda el resultado”).
- **Stage (Etapa):** Spark divide un Job en sub-etapas. ¿Cuándo corta una etapa y empieza otra? Cuando ocurre un **Shuffle** (es decir, cuando los Nodos/Executors necesitan pasarse datos por red entre ellos, como en un `GROUP BY`).
- **Task (Tarea):** La unidad más pequeña de trabajo. Un *Executor* ejecuta una *Task* sobre una *Partición* de datos. ¡A más particiones, más tareas paralelas!
- **DAG (Grafo Acíclico Dirigido):** Es el mapa estricto de pasos que Spark dibuja internamente antes de ejecutar nada para calcular la ruta más óptima.
- **Dependencia Estrecha (Narrow):** Una tarea necesita datos que ya están en el *mismo* nodo (Rápido).
- **Dependencia Ancha (Wide):** Una tarea necesita datos esparcidos por *varios* nodos, obligando a mover datos por la red (Shuffle - Lento).

1.3.5 1.6 APIs de Spark: Bajo vs Alto Nivel

A lo largo de su evolución (desde 2014 hasta hoy), la forma de programar en Spark ha cambiado drásticamente.

1. **API Estructurada (Alto Nivel): DataFrames y SQL**
 - **Es el estándar absoluto hoy en día (Spark 3.x y 4.x).**
 - Un DataFrame es como una tabla estructurada (filas y columnas), muy similar a usar la librería *pandas* o escribir sentencias SQL.
 - **Ventaja:** Spark sabe qué estructura tienen los datos y tiene un optimizador interno mágico llamado *Catalyst* que reescribe tu código para que vuele.
 - **Casos de uso:** El 95% del tiempo. Transformaciones, ETL, lectura/escritura de HDFS/Parquet/CSVs.
2. **API no estructurada (Bajo Nivel): RDDs (Resilient Distributed Datasets)**

- Es la estructura original de Spark. Colecciones de código distribuido sin estructura definida (orientado a objetos).
- **Ventaja:** Tienes el control absoluto de bajo nivel sobre cada registro y partición.
- **Desventaja:** Spark no puede optimizarlo tan bien como un DataFrame, y es más engorroso de programar.
- **Casos de uso:** Rara vez (5%). Solo si trabajas con estructuras sumamente exóticas o lógicas mega-personalizadas que no encajen en una tabla.

1.3.6 1.7 El Cerebro de la Ejecución Distribuida

Para entender por qué tu código vuela o por qué colapsa la memoria, es clave entender este vocabulario:

- **Job (Trabajo):** La gran tarea completa que Spark debe hacer (ej: “Lee este Parquet, agrúpalo y guarda el resultado”).
- **Stage (Etapa):** Spark divide un Job en sub-etapas. ¿Cuándo corta una etapa y empieza otra? Cuando ocurre un **Shuffle** (es decir, cuando los Nodos/Executors necesitan pasarse datos por red entre ellos, como en un gran **GROUP BY**).
- **Task (Tarea):** La unidad más pequeña de trabajo. Un *Executor* ejecuta una *Task* sobre una *Partición* de datos. ¡A más particiones, más tareas paralelas!
- **DAG (Grafo Acíclico Dirigido):** Es el mapa estricto de pasos que Spark dibuja internamente antes de ejecutar nada para calcular la ruta más óptima.
- **Dependencia Estrecha (Narrow):** Una tarea necesita datos que ya están en el *mismo* nodo (Rápido).
- **Dependencia Ancha (Wide):** Una tarea necesita datos esparcidos por *varios* nodos, obligando a mover datos por la red (Shuffle - Lento).

```
[1]: from pyspark.sql import SparkSession

# Inicializamos la sesión de Spark.
# master("local[*]") le dice a Spark que use todos los núcleos disponibles de
# NUESTRO contenedor.
spark = SparkSession.builder \
    .appName("HolaHadoopSpark") \
    .master("local[*]") \
    .getOrCreate()

print(" Spark Initialized!")
print(f"Spark Version: {spark.version}")
```

```
Spark Initialized!
Spark Version: 3.5.0
```

1.3.7 1. Crear datos en memoria distribuidos (DataFrame)

Spark usa **DataFrames**, que son muy similares a los de Pandas, pero preparados para distribuirse en muchos ordenadores bajo el capó (RDBMS).

```
[2]: # Creamos algunos datos en memoria
data = [
    ("Alice", 28, "Ingeniería"),
    ("Bob", 35, "Ventas"),
    ("Carlos", 42, "Marketing"),
    ("Diana", 31, "Ingeniería")
]
columns = ["Nombre", "Edad", "Departamento"]

# Lo convertimos a un Spark DataFrame
df = spark.createDataFrame(data, schema=columns)

# Mostramos el contenido (evaluación perezosa, se ejecuta ahora)
df.show()
```

```
+-----+-----+-----+
|Nombre|Edad|Departamento|
+-----+-----+-----+
| Alice|  28|  Ingeniería|
|  Bob|  35|    Ventas|
|Carlos|  42|  Marketing|
| Diana|  31|  Ingeniería|
+-----+-----+-----+
```

1.3.8 2. Procesamiento (Transformaciones)

Vamos a decirle a Spark que nos diga el promedio de edad por departamento. Fíjate qué parecido es a Pandas o SQL.

```
[3]: import pyspark.sql.functions as F

# Agrupamos por departamento y sacamos la media de edad
df_resumen = df.groupBy("Departamento").agg(F.avg("Edad").alias("Edad_Media"))

df_resumen.show()
```

```
+-----+-----+
|Departamento|Edad_Media|
+-----+-----+
|  Ingeniería|      29.5|
|    Ventas|      35.0|
|  Marketing|      42.0|
+-----+-----+
```

1.3.9 3. Integración con Hadoop (HDFS)

La verdadera magia ocurre al guardar en HDFS. Le diremos a nuestro *Spark Local* que se conecte con el *Namenode* del Clúster Hadoop usando el protocolo `hdfs://namenode:9000`

Nota: Al escribir en HDFS con Spark, a menudo no se crea un solo archivo, sino una carpeta (ej. empleados.parquet) con varios trozos de archivos adentro, lo cual es ideal para lectura paralela.

```
[4]: hdfs_path = "hdfs://namenode:9000/user/alumnos/spark_empleados"

# 1. Guardamos el DataFrame procesado en formato Parquet (el formato rey de Big_
↳Data)
df_resumen.write.mode("overwrite").parquet(hdfs_path)

print(f" Datos guardados con éxito en HDFS en la ruta: {hdfs_path}")
```

Datos guardados con éxito en HDFS en la ruta:
`hdfs://namenode:9000/user/alumnos/spark_empleados`

Podemos verificar desde el propio cuaderno que Spark ha creado esa carpeta en HDFS lanzando un comando de sistema `hdfs`:

```
[5]: !hdfs dfs -ls /user/alumnos/spark_empleados
```

```
Found 2 items
-rw-r--r--    2 jovyan supergroup          0 2026-03-11 17:38
/user/alumnos/spark_empleados/_SUCCESS
-rw-r--r--    2 jovyan supergroup      818 2026-03-11 17:38 /user/alumnos/spark
_empleados/part-00000-2451e900-4819-42a3-bab3-0e0ecc3812d4-c000.snappy.parquet
```

1.3.10 4. Lectura desde HDFS

Por último, simulamos el proceso de lectura. Esto es lo que harías al inicio de un proyecto de Machine Learning: recuperar datos masivos guardados en Hadoop para procesarlos en Spark.

```
[6]: # Spark lee automáticamente todos los trozos dentro de la carpeta parqueada
df_recuperado = spark.read.parquet(hdfs_path)

print(" Datos recuperados de HDFS:")
df_recuperado.show()
```

Datos recuperados de HDFS:

-----+-----
Departamento Edad_Media
-----+-----
Ingeniería 29.5
Ventas 35.0
Marketing 42.0
-----+-----

```
[7]: # Siempre es una buena práctica cerrar la sesión de Spark cuando terminas ↵
      ↪recursos.
spark.stop()
print(" Sesión de Spark detenida.")
```

Sesión de Spark detenida.

1.3.11 Recursos Oficiales y Novedades (Spark Moderno)

Si quieres seguir explorando Apache Spark en casa o prepararte para entornos de producción modernos (o instalarlo tú mismo), aquí tienes los recursos oficiales más actualizados:

- **Instalación Rápida Local (Python):** En vez de usar Docker, puedes instalar Spark localmente para pruebas usando pip. `bash pip install "pyspark>=4.1,<4.2"` (*Nota histórica: Aunque originalmente Spark soportaba Scala, Java, Python y R; R está siendo paulatinamente deprecado en las documentaciones recientes de la rama 4.x. Hoy en día, Python (PySpark) y Scala son los reyes absolutos*).
 - **Spark Connect:** Es una nueva arquitectura cliente-servidor introducida recientemente en Spark. Permite que aplicaciones súper ligeras (como tu IDE o un micro-script de Python) se conecten a un clúster remoto de Spark sin tener que empaquetar todo el motor pesado internamente.
 - **Documentación General Oficial:** <https://spark.apache.org/docs/latest/>
 - **Referencia de API para DataFrames (PySpark):** <https://spark.apache.org/docs/latest/api/python/index.html>
-

1.3.12 Recursos Oficiales y Novedades (Spark Moderno)

Si quieres seguir explorando Apache Spark en casa o prepararte para entornos de producción modernos (o instalarlo tú mismo), aquí tienes los recursos oficiales más actualizados:

- **Instalación Rápida Local (Python):** En vez de usar Docker, puedes instalar Spark localmente para pruebas usando pip. `bash pip install "pyspark>=4.1,<4.2"` (*Nota histórica: Aunque originalmente Spark soportaba Scala, Java, Python y R; R está siendo paulatinamente deprecado en las documentaciones recientes de la rama 4.x. Hoy en día, Python (PySpark) y Scala son los reyes absolutos*).
- **Spark Connect:** Es una nueva arquitectura cliente-servidor introducida recientemente en Spark. Permite que aplicaciones súper ligeras (como tu IDE o un micro-script de Python) se conecten a un clúster remoto de Spark sin tener que empaquetar todo el motor pesado internamente.
- **Documentación General Oficial:** <https://spark.apache.org/docs/latest/>
- **Referencia de API para DataFrames (PySpark):** <https://spark.apache.org/docs/latest/api/python/index.html>

03b_Spark_ETL_Basico

March 11, 2026

1 Práctica 0: Introducción a la Arquitectura Medallón con Spark

Dataset: Supermarket Sales (Ligero, ~1000 filas) **Tecnología:** Apache Spark (Modo Local) sobre HDFS

Este cuaderno ilustra el flujo de trabajo completo de Ingeniería de Datos utilizando la **Arquitectura Medallón** (Medallion Architecture). Es el estándar de la industria para organizar los lagos de datos (Data Lakes) en tres capas lógicas de madurez:

- **Capa Bronce (Raw):** Ingesta (Extract & Load). Contiene los datos crudos originales, tal cual vienen de la fuente.
- **Capa Plata (Silver):** Transformación Principal. Datos limpios, estandarizados, sin nulos y con tipos correctos, guardados en formato columnar (Parquet).
- **Capa Oro (Gold):** Lógica de Negocio. Transformación de negocio (agregados, KPIs, modelos para BI).

Objetivos: 1. Ingestar datos crudos a la Capa Bronce en formato CSV. 2. Limpiar e inferir el esquema para guardar los datos listos en la Capa Plata. 3. Calcular los ingresos totales por línea de producto y guardarlos en la Capa Oro.

```
[1]: # 1. Iniciar Sesión Spark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, desc

spark = SparkSession.builder \
    .appName("Practica_Supermercado_Spark_Medallion") \
    .master("local[*]") \
    .getOrCreate()

print("Spark inicializado y listo para trabajar en capas.")
```

Spark inicializado y listo para trabajar en capas.

1.0.1 Capa Bronce (Raw): Ingesta de Datos (E + L) y Transformación Mínima

Vamos a descargar un archivo CSV crudo con el histórico de ventas (usando Python básico para evitar bloqueos) y lo guardaremos intacto en nuestro lago de datos HDFS.

```
[2]: import urllib.request
import os
```

```
# Descargamos directamente desde un repositorio de Github Público para evitar
↳ errores 403 de Kaggle
# https://github.com/sushantag9/Supermarket-Sales-Data-Analysis/blob/master/
↳ supermarket_sales%20-%20Sheet1.csv
url = "https://raw.githubusercontent.com/sushantag9/
↳ Supermarket-Sales-Data-Analysis/master/supermarket_sales%20-%20Sheet1.csv"
os.makedirs("supermarket_sales", exist_ok=True)
archivo_local = "supermarket_sales/sales.csv"

urllib.request.urlretrieve(url, archivo_local)
print("¡Descarga finalizada con éxito!")
```

¡Descarga finalizada con éxito!

```
[3]: # Ingesta a la Capa Bronce distribuida en HDFS
!hdfs dfs -mkdir -p /data/bronze/supermarket/
!hdfs dfs -put -f supermarket_sales/*.csv /data/bronze/supermarket/
```

1.0.2 Capa Plata (Silver): Transformación Principal

Leemos el archivo crudo desde la capa Bronce, realizamos la estandarización (inferir esquema), limpiamos valores nulos, y lo guardamos comprimido de forma ultra-eficiente (Parquet) para las futuras analíticas.

```
[4]: # 1. Leemos de Bronce (CSV)
ruta_bronce = "hdfs://namenode:9000/data/bronze/supermarket/*.csv"
df_bronce = spark.read.option("header", "true").option("inferSchema", "true").
↳ csv(ruta_bronce)
print("Esquema bronze")
df_bronce.printSchema()

# 2. Transformación principal: limpieza, validación y estandarización de
↳ columnas
df_plata_casi_lista = df_bronce.na.drop()
# Mantenemos consistencia removiendo espacios en los nombres de columnas
df_plata = df_plata_casi_lista.withColumnRenamed("Product line",
↳ "product_line").withColumnRenamed("Total", "total_sales")

# 3. Guardamos en la Capa Plata formateado nativamente para Big Data (Parquet)
ruta_plata = "hdfs://namenode:9000/data/silver/supermarket/"
df_plata.write.mode("overwrite").parquet(ruta_plata)
print("Esquema planta")
df_plata.printSchema()
```

Esquema bronze
root

```

|-- Invoice ID: string (nullable = true)
|-- Branch: string (nullable = true)
|-- City: string (nullable = true)
|-- Customer type: string (nullable = true)
|-- Gender: string (nullable = true)
|-- Product line: string (nullable = true)
|-- Unit price: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Tax 5%: double (nullable = true)
|-- Total: double (nullable = true)
|-- Date: string (nullable = true)
|-- Time: timestamp (nullable = true)
|-- Payment: string (nullable = true)
|-- cogs: double (nullable = true)
|-- gross margin percentage: double (nullable = true)
|-- gross income: double (nullable = true)
|-- Rating: double (nullable = true)

```

Esquema planta

root

```

|-- Invoice ID: string (nullable = true)
|-- Branch: string (nullable = true)
|-- City: string (nullable = true)
|-- Customer type: string (nullable = true)
|-- Gender: string (nullable = true)
|-- product_line: string (nullable = true)
|-- Unit price: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Tax 5%: double (nullable = true)
|-- total_sales: double (nullable = true)
|-- Date: string (nullable = true)
|-- Time: timestamp (nullable = true)
|-- Payment: string (nullable = true)
|-- cogs: double (nullable = true)
|-- gross margin percentage: double (nullable = true)
|-- gross income: double (nullable = true)
|-- Rating: double (nullable = true)

```

1.0.3 Capa Oro (Gold): Transformación de Negocio

En esta capa leemos los datos ya estandarizados de la capa Plata y creamos agregaciones u operaciones complejas. Estos son los datos finales (Data Marts) que consumiría tu equipo de Business Intelligence (PowerBI/Tableau).

```

[5]: # 1. Leemos directamente de la Capa Plata (Nota: leer un Parquet es rapidísimo)
df_silver_ready = spark.read.parquet(ruta_plata)

```

```

# 2. Convertimos el detalle transaccional en un modelo agregado (KPI): ¿Qué
↳ línea de productos genera más ingresos totales?
df_oro = df_silver_ready.groupBy("product_line").agg(sum("total_sales").
↳ alias("ingresos_totales_kpi")).orderBy(desc("ingresos_totales_kpi"))

# 3. Guardamos nuestro KPI final en la Capa Oro listos para consumo
ruta_oro = "hdfs://namenode:9000/data/gold/kpi_ventas_producto/"
df_oro.write.mode("overwrite").parquet(ruta_oro)

df_oro.show()

```

```

+-----+-----+
|      product_line|ingresos_totales_kpi|
+-----+-----+
| Food and beverages| 56144.844000000005|
| Sports and travel| 55122.826499999996|
|Electronic access...| 54337.531500000005|
| Fashion accessories|          54305.895|
| Home and lifestyle| 53861.913000000001|
| Health and beauty| 49193.739000000016|
+-----+-----+

```

```
[6]: spark.stop()
```

```
[ ]:
```

04_Spark_Practica_Ventas

March 12, 2026

1 Práctica 1: E-Commerce y Agregaciones Masivas en Capas

Dataset: Ecommerce Behavior Data (Millones de interacciones) **Tecnología:** Apache Spark (Modo Local) sobre HDFS

En esta práctica vamos a procesar varios millones de registros reales aplicando la **Arquitectura Medallón**. El dataset se compone de **múltiples archivos CSV** a la vez.

¡ATENCIÓN! - Posible Error 403 Forbidden de Kaggle > Si al ejecutar la celda de Kaggle te da un `HTTPError: 403 Client Error: Forbidden`, significa que tu archivo `kaggle.json` no tiene credenciales válidas, o que has superado el límite de descargas de la API. ¡Asegúrate de haber generado un nuevo token en Kaggle.com!

```
[1]: from pyspark.sql import SparkSession
      from pyspark.sql.functions import col, sum, count, desc, when, trim
```

```
[2]: spark = (SparkSession.builder
              .appName("Practica_Ventas_Kaggle")
              .master("local[*]")
              .config("spark.driver.memory", "4g")
              .config("spark.sql.shuffle.partitions", "16")
              .getOrCreate()
              )

# Ajusta particiones para local (muy importante)
# En local, el default puede ser malo (demasiadas o muy pocas tareas).
# Si tienes 8 núcleos (ejemplo)
# spark = (
#     SparkSession.builder
#     .appName("Intro_Spark")
#     .master("local[*]")
#     .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
#     .getOrCreate()
# )
# Regla práctica
# dataset grande + local → prueba 8 / 16 / 32
# - si pones demasiado alto → overhead
# - si pones demasiado bajo → poco paralelismo
```

```
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
    ↪")
```

Versión de PySpark 3.5.0.

1.0.1 Bronce: Ingesta Masiva

Este dataset es gigante (varios GB) y contiene archivos CSV que representan diferentes meses (Octubre, Noviembre...). HDFS brillará aquí guardándolos distribuidos, y Spark los leerá todos a la vez.

```
[3]: !pip install -q opendatasets
import opendatasets as od

# Dataset masivo de comportamiento en Ecommerce
dataset_url = "https://www.kaggle.com/datasets/mkechinov/
    ↪ecommerce-behavior-data-from-multi-category-store"

od.download(dataset_url)

# Ingestamos TODOS los CSV (*.csv) a una única ruta en Bronce.
!hdfs dfs -mkdir -p /data/bronze/ecommerce/
!hdfs dfs -put -f ecommerce-behavior-data-from-multi-category-store/*.csv /data/
    ↪bronze/ecommerce/
```

Please provide your Kaggle credentials to download this dataset. Learn more:

<http://bit.ly/kaggle-creds>

Your Kaggle username:

antonioferrermartnez

Your Kaggle Key:

.....

Dataset URL: <https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store>

Downloading ecommerce-behavior-data-from-multi-category-store.zip to
./ecommerce-behavior-data-from-multi-category-store

100%| | 4.29G/4.29G [13:24<00:00, 5.73MB/s]

1.0.2 Plata: Estandarización

Uniremos todos los CSVs y los validaremos (quitando basuras/nulos) para convertirlos a Parquet.

```
[4]: ruta_bronze = "hdfs://namenode:9000/data/bronze/ecommerce/*.csv"

# 1. Leer un directorio entero de CSV con millones de filas haciendo que
#     adivine el esquema
df_ventas = spark.read.option("header", "true").option("inferSchema", "true").
    csv(ruta_bronze)

total = df_ventas.count()
print(f"Total de registros: {total:,}")
```

Total de registros: 109,950,743

```
[5]: # VER SI ALGUN ID TIENE NULOS
# Ver una sola columna
print("user_id nulos:", df_ventas.filter(col("user_id").isNull()).count())
# Ver varias a la vez:
id_cols = ["product_id", "category_id", "user_id", "user_session"]

df_ventas.select([
    count(when(col(c).isNull(), c)).alias(f"{c}_nulls")
    for c in id_cols
]).show()

# Extra: detectar nulos o vacíos (strings)
df_ventas.filter(
    col("user_session").isNull() | (trim(col("user_session")) == "")
).show(20, truncate=False)
```

user_id nulos: 0

```
+-----+-----+-----+-----+
|product_id_nulls|category_id_nulls|user_id_nulls|user_session_nulls|
+-----+-----+-----+-----+
|              0|              0|              0|              12|
+-----+-----+-----+-----+
```



```
+-----+-----+-----+-----+-----+
|event_time      |event_type|product_id|category_id      |category_code
|brand |price |user_id |user_session|
+-----+-----+-----+-----+-----+
|2019-11-09 15:32:27|cart      |19700004  |2053013559104766575|NULL
|kabrita|37.77  |539704497|NULL              |
|2019-11-09 17:15:24|cart      |1005083   |
|2053013555631882655|electronics.smartphone      |honor  |566.27 |568843390|NULL
|
|2019-11-13 04:02:03|cart      |4804008   |
```

```

|2053013554658804075|electronics.audio.headphone|bluedio|97.81 |570411102|NULL
|
|2019-11-13 07:18:35|cart |1004767
|2053013555631882655|electronics.smartphone |samsung|243.51 |570878749|NULL
|
|2019-11-23 12:53:50|cart |7600528 |2053013552821698803|NULL
|tp-link|16.73 |575357602|NULL |
|2019-11-25 05:04:42|cart |1802104
|2053013554415534427|electronics.video.tv |arg |360.09 |573722572|NULL
|
|2019-11-25 07:03:38|cart |21403753
|2053013561579406073|electronics.clocks |NULL |181.47 |576301354|NULL
|
|2019-11-26 07:48:12|cart |4804718
|2053013554658804075|electronics.audio.headphone|apple |334.58 |576935861|NULL
|
|2019-11-27 07:02:20|cart |12719553 |2053013553559896355|NULL
|NULL |55.6 |577167045|NULL |
|2019-11-29 17:25:56|cart |1005116
|2053013555631882655|electronics.smartphone |apple |1001.42|579123407|NULL
|
|2019-10-06 14:26:10|cart |1801723
|2053013554415534427|electronics.video.tv |tcl |135.65 |557388939|NULL
|
|2019-10-25 10:36:14|cart |1004767
|2053013555631882655|electronics.smartphone |samsung|246.52 |549825742|NULL
|
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+

```

```
[6]: df_ventas.show(5)
```

```

+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
|          event_time|event_type|product_id|          category_id|
category_code| brand| price| user_id|          user_session|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
|2019-11-01 00:00:00|      view|  1003461|2053013555631882655|electronics.smart
...|xiaomi|489.07|520088904|4d3b30da-a5e4-49d...|
|2019-11-01 00:00:00|      view|  5000088|2053013566100866035|appliances.sewing
...|janome|293.65|530496790|8e5f4f83-366c-4f7...|
|2019-11-01 00:00:01|      view|  17302664|2053013553853497655|
NULL| creed| 28.31|561587266|755422e7-9040-477...|
|2019-11-01 00:00:01|      view|
3601530|2053013563810775923|appliances.kitche...|
lg|712.87|518085591|3bfb58cd-7892-48c...|

```

```
|2019-11-01 00:00:01|      view|    1004775|2053013555631882655|electronics.smart
...|xiaomi|183.27|558856683|313628f1-68b8-460...|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 5 rows
```

```
[7]: # 2. Transformación principal: Filtra las interacciones de compras defectuosas
      ↪(precios nulos) y eventos que no sean compras
df_plata = df_ventas.na.drop(subset=["price", "brand"]).
      ↪filter(col("event_type") == "purchase")
df_plata.show(5)
```

```
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
|      event_time|event_type|product_id|      category_id|
category_code|    brand| price|  user_id|      user_session|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
|2019-11-01 00:01:04|  purchase|
1005161|2053013555631882655|electronics.smart...|
xiaomi|211.92|513351129|e6b7ce9b-1938-4e2...|
|2019-11-01 00:04:51|  purchase|
1004856|2053013555631882655|electronics.smart...|
samsung|128.42|562958505|0f039697-fedc-40f...|
|2019-11-01 00:05:34|  purchase|  26401669|2053013563651392361|
NULL|  lucente|109.66|541854711|c41c44d5-ef9b-41b...|
|2019-11-01 00:06:33|  purchase|
1801881|2053013554415534427|electronics.video.tv|  samsung|
488.8|557746614|4d76d6d3-fff5-488...|
|2019-11-01 00:06:34|  purchase|   5800823|2053013553945772349|electronics.audio
...|nakamichi|123.56|514166940|8ef5214a-86ad-4d0...|
+-----+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 5 rows
```

```
[8]: # 3. Escribir a Plata
ruta_silver = "hdfs://namenode:9000/data/silver/ecommerce/"
df_plata.repartition(16).write.mode("overwrite").parquet(ruta_silver)
print("Escritura a plata realizada")
```

Escritura a plata realizada

1.0.3 Oro: Transformaciones de Negocio (KPIs)

```
[9]: # 1. Leemos de nuestra Plataforma Silver
df_silver = spark.read.parquet(ruta_silver)

# 2. Regla de Negocio: ¿Qué marca (brand) ha facturado más dinero sumando todos
    ↪ los meses?
df_top_ventas = df_silver.groupBy("brand").agg(sum("price").
    ↪ alias("facturacion_total")).orderBy(desc("facturacion_total"))

# 3. Servir a BI
ruta_gold = "hdfs://namenode:9000/data/gold/top_ventas_ecommerce/"
df_top_ventas.write.mode("overwrite").parquet(ruta_gold)
df_top_ventas.show(10)
```

```
+-----+-----+
| brand| facturacion_total|
+-----+-----+
| apple|2.3872179370001447E8|
|samsung|1.0127741348000082E8|
| xiaomi| 2.04538992500001E7|
| huawei| 9664104.08999993|
| lg| 8626906.719999976|
| acer| 6924026.049999971|
|lucente| 6651658.939999944|
| sony| 6341082.980000027|
| oppo| 5901500.520000067|
| lenovo| 4450744.829999997|
+-----+-----+
```

only showing top 10 rows

1.0.4 1) Usa un formato columnar (Parquet) en vez de CSV

Si estás leyendo CSV, ese suele ser el mayor cuello de botella.

- **CSV** = lento (parseo, tipos, texto)
- **Parquet** = mucho más rápido (columnar, comprimido, schema)

```
[10]: # Ejemplo filtro de nulos sobre df_silver
id_cols = ["product_id", "category_id", "user_id", "user_session"]

df_silver.select([
    count(when(col(c).isNull(), c)).alias(f"{c}_nulls")
    for c in id_cols
]).show()
```

```
+-----+-----+-----+-----+
```

product_id_nulls category_id_nulls user_id_nulls user_session_nulls
+-----+-----+-----+-----+
0 0 0 0
+-----+-----+-----+-----+

1.0.5 2) Define esquema manualmente (no inferSchema=True)

inferSchema en archivos grandes hace una pasada extra y puede ser lento.

Mejor:

```
[11]: from pyspark.sql.types import *

schema = StructType([
    StructField("event_time", StringType(), True),
    StructField("event_type", StringType(), True),
    StructField("product_id", LongType(), True),
    StructField("category_id", LongType(), True),
    StructField("category_code", StringType(), True),
    StructField("brand", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("user_id", LongType(), True),
    StructField("user_session", StringType(), True),
])

ruta_bronze = "hdfs://namenode:9000/data/bronze/ecommerce/*.csv"

# 1. Leer un directorio entero de CSV con millones de filas haciendo que
#    adivine el esquema
# df_ventas = spark.read.option("header", "true").option("inferSchema", "true").
#    csv(ruta_bronze)

df = spark.read.csv(ruta_bronze, header=True, schema=schema)
df.show(10)
```

+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+
event_time event_type product_id category_id
category_code brand price user_id user_session
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+
2019-11-01 00:00:... view
1003461 2053013555631882655 electronics.smart...
xiaomi 489.07 520088904 4d3b30da-a5e4-49d...
2019-11-01 00:00:... view
5000088 2053013566100866035 appliances.sewing...
janome 293.65 530496790 8e5f4f83-366c-4f7...
2019-11-01 00:00:... view 17302664 2053013553853497655

```

NULL| creed| 28.31|561587266|755422e7-9040-477...|
|2019-11-01 00:00:...| view|
3601530|2053013563810775923|appliances.kitchen...|
lg|712.87|518085591|3bfb58cd-7892-48c...|
|2019-11-01 00:00:...| view|
1004775|2053013555631882655|electronics.smart...|
xiaomi|183.27|558856683|313628f1-68b8-460...|
|2019-11-01 00:00:...| view| 1306894|2053013558920217191|
computers.notebook| hp|360.09|520772685|816a59f3-f5ae-4cc...|
|2019-11-01 00:00:...| view| 1306421|2053013558920217191|
computers.notebook| hp|514.56|514028527|df8184cc-3694-454...|
|2019-11-01 00:00:...| view| 15900065|2053013558190408249|
NULL| rondell| 30.86|518574284|5e6ef132-4d7c-473...|
|2019-11-01 00:00:...| view| 12708937|2053013553559896355|
NULL|michelin| 72.72|532364121|0a899268-31eb-46d...|
|2019-11-01 00:00:...| view|
1004258|2053013555631882655|electronics.smart...|
apple|732.07|532647354|d2d3d2c6-631d-489...|
+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
only showing top 10 rows

```

```

[12]: # Muestra cuántas particiones tiene actualmente el DataFrame.
      # Esto te ayuda a entender cómo Spark dividirá el trabajo en tareas paralelas.
      print(df.rdd.getNumPartitions())

      # Reorganiza el DataFrame para que tenga exactamente 16 particiones.
      # Spark redistribuye los datos (shuffle), lo que puede mejorar el paralelismo
      # si antes había muy pocas particiones o estaban mal distribuidas.
      df = df.repartition(16)

```

110

1.1 Explicación

1.1.1 print(df.rdd.getNumPartitions())

- Devuelve el **número actual de particiones** del DataFrame.
- En Spark, las particiones son “trozos” de datos que se procesan en paralelo.
- Si tienes muy pocas, no aprovechas los cores.
- Si tienes demasiadas, hay overhead (muchas tareas pequeñas).

1.1.2 df = df.repartition(16)

- Fuerza al DataFrame a tener **16 particiones**.

- Esto puede mejorar rendimiento en operaciones posteriores (`groupBy`, `join`, etc.) si 16 encaja mejor con tu máquina.
- **Ojo:** `repartition()` suele implicar **shuffle** (mueve datos entre particiones), así que no conviene usarlo sin necesidad.

1.2 Cuándo usarlo

- Antes de operaciones pesadas si ves mala distribución / pocas particiones.
- Si quieres ajustar el paralelismo en local (por ejemplo 8, 16, 32).
- No lo uses en cada paso del pipeline, porque el shuffle cuesta tiempo.

1.3 Avanzado

```
[13]: from pyspark.sql import SparkSession
from pyspark import StorageLevel

# =====
# 1) Crear sesión de Spark (local)
# =====
spark = (
    SparkSession.builder
        .appName("Análisis_Local_Spark")
        .master("local[*]") # Usa todos los núcleos disponibles de tu PC
        .config("spark.driver.memory", "8g") # Memoria para Spark (ajusta según tu
        ↪ RAM)
        .config("spark.sql.shuffle.partitions", "16")
        # Número de particiones por defecto en operaciones pesadas (groupBy, join,
        ↪ orderBy)
        .config("spark.sql.adaptive.enabled", "true")
        # Spark puede ajustar particiones automáticamente en algunos casos (AQE)
        .getOrCreate()
)

# =====
# 2) Leer datos (ejemplo)
# =====
df = spark.read.parquet(ruta_silver)
# Mejor usar Parquet (más rápido que CSV)

# =====
# 3) Ver cuántas particiones tiene el DataFrame
# =====
print("Particiones actuales:", df.rdd.getNumPartitions())
# Esto te dice en cuántos "trozos" Spark dividirá el trabajo en paralelo
```

```

# =====
# 4) Ajustar particiones si hace falta
# =====
df = df.repartition(16)
# Reparte los datos en 16 particiones
# Útil si quieres mejorar paralelismo para operaciones pesadas
# Ojo: repartition hace shuffle (mueve datos), así que no lo uses muchas veces
# ↪ sin necesidad

print("Particiones después de repartition:", df.rdd.getNumPartitions())

# =====
# 5) Cachear solo si lo vas a reutilizar varias veces
# =====
df.persist(StorageLevel.MEMORY_AND_DISK)
# Guarda el DataFrame en memoria (y en disco si no cabe)
# Bueno si vas a hacer varios análisis sobre el mismo df

df.count()
# Acción para "materializar" el cache (sin esto, todavía no se guarda realmente)

# =====
# 6) Ejemplo de análisis
# =====
df.groupBy("event_type").count().show()
# groupBy suele hacer shuffle; por eso ayuda tener bien configuradas las
# ↪ particiones

```

Particiones actuales: 8

Particiones después de repartition: 16

```

+-----+-----+
|event_type|  count|
+-----+-----+
|  purchase|1528301|
+-----+-----+

```

1.4 Explicación fácil (qué hace cada cosa)

1.4.1 local[*]

Usa todos los núcleos de tu computador.

Más paralelo = mejor uso de tu CPU.

1.4.2 spark.driver.memory = 8g

Le da memoria a Spark.

Si tienes poca memoria, Spark va lento o puede fallar.

1.4.3 `spark.sql.shuffle.partitions = 16`

Controla cuántas particiones usa Spark en operaciones pesadas (`groupBy`, `join`, etc.).
En local, 16 suele ir mejor que 200 (que muchas veces es demasiado).

1.4.4 `df.rdd.getNumPartitions()`

Te dice cuántas particiones tiene el `DataFrame`.
Sirve para entender si estás usando pocas o demasiadas.

1.4.5 `df.repartition(16)`

Reorganiza los datos en 16 particiones.
Ayuda a repartir mejor el trabajo, pero cuesta (`shuffle`).

1.4.6 `persist(MEMORY_AND_DISK)`

Guarda el `DataFrame` para reutilizarlo varias veces.
Evita recalcular todo desde cero en cada operación.

1.5 Regla fácil para recordar

- **Explorar rápido** → usa muestra (`sample`)
- **Operaciones pesadas** → revisa particiones
- **Repetir análisis** → usa `cache/persist`
- **Velocidad de lectura** → usa **Parquet**

```
[14]: # Cerrar la sesión de Spark al finalizar
try:
    spark.stop()
    print("SparkSession detenida.")
except Exception as e:
    print("No se pudo detener SparkSession:", e)
```

SparkSession detenida.

```
[ ]:
```

EJERCICIO 3

Spark Avanzado, Rendimiento e Integridad

05_Spark_Practica_Taxis_NYC

March 12, 2026

1 Práctica 2: Taxis de NYC - Fechas y Ventanas (Arquitectura Medallón)

Dataset: NYC Taxi Trip Duration **Tecnología:** Apache Spark (Modo Local)

El dataset de Taxis de Nueva York es un clásico. Descargaremos una versión manejable de la competición de Kaggle para probar el manejo de **Timestamps** y aplicar **Window Functions** avanzadas de Spark para crear cuadros de mando (Oro).

Recuerda: Si recibes Error 403 Forbidden al descargar, revisa tus credenciales temporales de `kaggle.json`.

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, hour, dayofweek, max, rank
from pyspark.sql.window import Window

spark = (SparkSession.builder
        .appName("Practica_Taxis_NYC_Medallion")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
        .getOrCreate()
        )

# spark = (
#     SparkSession.builder
#     .appName("Intro_Spark")
#     .master("local[*]")
#     .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
#     .getOrCreate()
# )
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
    ↪")
```

Versión de PySpark 3.5.0.

1.0.1 Bronze: Ingesta

```
[2]: !pip install -q opendatasets
import opendatasets as od

dataset_url = "https://www.kaggle.com/datasets/yasserh/nyc-taxi-trip-duration"
od.download(dataset_url)

# Ingestamos en Crudo
!hdfs dfs -mkdir -p /data/bronze/taxi/
!hdfs dfs -put -f nyc-taxi-trip-duration/*.csv /data/bronze/taxi/
```

Please provide your Kaggle credentials to download this dataset. Learn more:

<http://bit.ly/kaggle-creds>

Your Kaggle username:

antonioferrermartnez

Your Kaggle Key:

• • • • •

Dataset URL: <https://www.kaggle.com/datasets/yasserh/nyc-taxi-trip-duration>

```
Downloading nyc-taxi-trip-duration.zip to ./nyc-taxi-trip-duration
```

100% | 63.9M/63.9M [00:09<00:00, 6.92MB/s]

1.0.2 Plata: Validaciones y Transformaciones Temporales

La columna `pickup_datetime` guarda la fecha y hora. En la capa de Plata, los Data Engineers se encargan de **enriquecer** los datos para que negocio lo tenga fácil.

```
[3]: ruta_bronze = "hdfs://namenode:9000/data/bronze/taxi/*.csv"
df_crudo = spark.read.option("header", "true").option("inferSchema", "true").
    .csv(ruta_bronze)
print("Carga de df_crudo")
```

Carga de df_crudo

```
[4]: df_crudo.show(1, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|id          |vendor_id|pickup_datetime   |dropoff_datetime
|passenger_count|pickup_longitude |pickup_latitude   |dropoff_longitude
|dropoff_latitude |store_and_fwd_flag|trip_duration|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+
```

```
|id2875421|2          |2016-03-14 17:24:55|2016-03-14 17:32:30|1
|-73.9821548461914|40.76793670654297|-73.96463012695312|40.765602111816406|N
|455      |
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+-----+
only showing top 1 row
```

```
[5]: df_crudo.show(1, truncate=False, vertical=True)
```

```
-RECORD 0-----
id          | id2875421
vendor_id   | 2
pickup_datetime | 2016-03-14 17:24:55
dropoff_datetime | 2016-03-14 17:32:30
passenger_count | 1
pickup_longitude | -73.9821548461914
pickup_latitude | 40.76793670654297
dropoff_longitude | -73.96463012695312
dropoff_latitude | 40.765602111816406
store_and_fwd_flag | N
trip_duration | 455
only showing top 1 row
```

```
[6]: df_crudo.printSchema()
```

```
root
 |-- id: string (nullable = true)
 |-- vendor_id: integer (nullable = true)
 |-- pickup_datetime: timestamp (nullable = true)
 |-- dropoff_datetime: timestamp (nullable = true)
 |-- passenger_count: integer (nullable = true)
 |-- pickup_longitude: double (nullable = true)
 |-- pickup_latitude: double (nullable = true)
 |-- dropoff_longitude: double (nullable = true)
 |-- dropoff_latitude: double (nullable = true)
 |-- store_and_fwd_flag: string (nullable = true)
 |-- trip_duration: integer (nullable = true)
```

```
[7]: # 1. Enriquecer los datos derivando dos columnas: el Día de la semana, y la
      ↪ Hora exacta.
df_plata = df_crudo.withColumn("dia_semana", dayofweek(col("pickup_datetime")))\
      ↪
      .withColumn("hora_recogida", hour(col("pickup_datetime")))
```

```
# dayofweek : Extract the day of the week of a given date/timestamp as integer.
↳ Ranges from 1 for a Sunday through to 7 for a Saturday
```

```
[8]: df_plata.show(1, truncate=False, vertical=True)
```

```
-RECORD 0-----
id                | id2875421
vendor_id         | 2
pickup_datetime   | 2016-03-14 17:24:55
dropoff_datetime  | 2016-03-14 17:32:30
passenger_count   | 1
pickup_longitude  | -73.9821548461914
pickup_latitude   | 40.76793670654297
dropoff_longitude | -73.96463012695312
dropoff_latitude  | 40.765602111816406
store_and_fwd_flag | N
trip_duration     | 455
dia_semana        | 2
hora_recogida     | 17
only showing top 1 row
```

```
[9]: df_plata.select("id", "vendor_id",
↳ "passenger_count", "dia_semana", "hora_recogida").show(10, truncate=False)
```

```
+-----+-----+-----+-----+-----+
|id      |vendor_id|passenger_count|dia_semana|hora_recogida|
+-----+-----+-----+-----+-----+
|id2875421|2        |1              |2         |17           |
|id2377394|1        |1              |1         |0            |
|id3858529|2        |1              |3         |11           |
|id3504673|2        |1              |4         |19           |
|id2181028|2        |1              |7         |13           |
|id0801584|2        |6              |7         |22           |
|id1813257|1        |4              |6         |22           |
|id1324603|2        |1              |7         |7            |
|id1301050|1        |1              |6         |23           |
|id0012891|2        |1              |5         |21           |
+-----+-----+-----+-----+-----+
only showing top 10 rows
```

```
[10]: # 2. Guardar el trabajo limpio
ruta_silver = "hdfs://namenode:9000/data/silver/taxi/"
df_plata.write.mode("overwrite").parquet(ruta_silver)
```

1.0.3 Oro: Analítica Avanzada (Funciones de Ventana)

KPI a la vista: Tu jefe quiere saber todos los detalles técnicos referidos de “cuál fue el viaje de mayor duración PARA CADA UNO de los días de la semana”. Un `groupBy` destruiría el detalle. Usaremos una Ventana.

2 NOTA: QUIERO QUE ME EXPLIQUES PORQUE SE HACE UNA VENTANA Y LO ENTIENDAS Y QUE ME SEPAS EXPLICAR LO QUE HACE EL CODIGO SIGUIENTE

```
[11]: df_silver = spark.read.parquet(ruta_silver)

# 1. Definimos la ventana: El contexto de nuestra búsqueda está particionado/
#     dentro de "cada dia_semana"
ventana_por_dia = Window.partitionBy("dia_semana").orderBy(col("trip_duration").
#     desc())
```

```
[12]: # 2. Aplicamos la ventana añadiendo un ranking a cada viaje e inyectando la
#     columna
df_ranking = df_silver.withColumn("ranking_duracion", rank().
#     over(ventana_por_dia))
df_ranking.select("dia_semana", "trip_duration", "passenger_count",
#     "pickup_datetime", "ranking_duracion").show(10, truncate=False)
```

dia_semana	trip_duration	passenger_count	pickup_datetime	ranking_duracion
5	86387	1	2016-06-30 16:37:52	1
5	86385	1	2016-06-23 16:01:45	2
5	86378	1	2016-05-12 13:48:19	3
5	86369	6	2016-05-26 14:55:11	4
5	86369	1	2016-02-25 15:17:29	4
5	86361	1	2016-04-14 17:20:48	6
5	86353	6	2016-06-09 20:52:30	7
5	86350	4	2016-06-02 21:47:46	8
5	86348	5	2016-05-26 13:18:44	9
5	86346	2	2016-03-03 22:02:24	10

only showing top 10 rows

```
[13]: # 3. Modelo de Negocio (Oro): Filtramos la medalla de oro de cada día (ranking
#     == 1)
df_oro = df_ranking.filter(col("ranking_duracion") == 1).select("dia_semana",
#     "trip_duration", "passenger_count", "pickup_datetime")
```

```
[14]: # 4. Publicar en Capa Oro
ruta_gold = "hdfs://namenode:9000/data/gold/taxi_longest_trips/"
df_oro.write.mode("overwrite").parquet(ruta_gold)
df_oro.show()
```

dia_semana	trip_duration	passenger_count	pickup_datetime
1	86369	1	2016-01-17 20:27:56
2	86392	2	2016-02-15 23:18:06
3	2227612	1	2016-01-05 06:14:15
4	86366	4	2016-03-09 02:59:46
5	86387	1	2016-06-30 16:37:52
6	86390	1	2016-05-06 00:00:10
7	3526282	1	2016-02-13 22:46:52

```
[15]: # Cerrar la sesión de Spark al finalizar
try:
    spark.stop()
    print("SparkSession detenida.")
except Exception as e:
    print("No se pudo detener SparkSession:", e)
```

SparkSession detenida.

06_Spark_Rendimiento_y_Shuffles

March 12, 2026

1 Práctica 3: Rendimiento, Shuffles y Joins (Medallón)

Dataset: US Accidents (Desde 2016, >7 Millones de filas) **Tecnología:** Apache Spark (Modo Local)

Los archivos CSV extraídos de base operativa en texto plano agotan procesadores y colapsan memorias. Apache inventó el formato **Parquet** que brilla entre tu capa Bronce y tu capa Plata.

Recuerda: Este dataset supera los 3GB. Tarda minutos en bajar de Kaggle (si falla, renueva tu token y evitar el Error 403).

```
[1]: import os
os.environ["SPARK_DRIVER_MEMORY"] = "4g"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python3"
from pyspark.sql import SparkSession

# Le damos bastante memoria a nuestro driver para no sufrir con 7.7 millones de
# ↪filas brutas
spark = (SparkSession.builder
        .appName("Practica_Rendimiento_Spark_Medallion")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "16")    # prueba 8, 16, 32
        .config("spark.driver.memory", "4g")
        .getOrCreate()
        )
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
    ↪")
```

Versión de PySpark 3.5.0.

1.0.1 Bronce (Extract)

```
[2]: !pip install -q opendatasets
import opendatasets as od

dataset_url = "https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents"
od.download(dataset_url)

!hdfs dfs -mkdir -p /data/bronze/accidents/
!hdfs dfs -put -f us-accidents/*.csv /data/bronze/accidents/
print("Descarga completada y carga a HDFS")
```

Please provide your Kaggle credentials to download this dataset. Learn more:

<http://bit.ly/kaggle-creds>

Your Kaggle username:

antonioferrermartnez

Your Kaggle Key:

.....

Dataset URL: <https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents>

Downloading us-accidents.zip to ./us-accidents

100%| | 653M/653M [01:19<00:00, 8.64MB/s]

Descarga completada y carga a HDFS

1.0.2 Plata: El Test de Velocidad (CSV vs Parquet)

La estandarización principal duele leerla de Bronce porque es CSV. Observa los tiempos.

```
[3]: import time
ruta_csv = "hdfs://namenode:9000/data/bronze/accidents/*.csv"
ruta_parquet = "hdfs://namenode:9000/data/silver/accidents/"

# --- 1. Mide tiempo de inferir el CSV por defecto (Masoquismo) ---
t0 = time.time()
df_csv = spark.read.option("header", "true").option("inferSchema", "true").
    ↪ csv(ruta_csv)
print(f"Líneas consumidas de Bronce: {df_csv.count()} (Tiempo bruto: {time.
    ↪ time() - t0:.2f} segundos)")

# --- 2. Convertimos el Dataset Crudo limpio a Plata (En bloques binarios,
    ↪ columnares) ---
# Usamos repartition(8) para distribuir la carga de memoria entre particiones
df_csv.repartition(8).write.mode("overwrite").parquet(ruta_parquet)

# --- 3. Mide tiempo de lectura de un Parquet nativo (La experiencia Plata) ---
```

```
t0 = time.time()
df_plata = spark.read.parquet(ruta_parquet)
print(f"Líneas consumidas de Plata: {df_plata.count()} (Tiempo veloz: {time.
↳time() - t0:.2f} segundos)")
```

Líneas consumidas de Bronce: 7728394 (Tiempo bruto: 41.68 segundos)

Líneas consumidas de Plata: 7728394 (Tiempo veloz: 1.05 segundos)

1.0.3 Oro: Shuffles Limitados y Particiones Controladas

Spark, por defecto divide los trabajos pesados de red de Oro (Shuffles, por ej un Group By para negocio) en **200 cajones (Particiones)**. A veces congestionará la red si lo hacemos mal.

```
[4]: # 1. Ajustamos la red
spark.conf.set("spark.sql.shuffle.partitions", "25")

# 2. Regla de Negocio (Oro): Agrupamos cruces de Estado y Gravedad (Lento pero
↳necesario)
df_oro = df_plata.groupBy("State", "Severity").count()

# 3. Optimizador Final: Hacemos un `coalesce` (fusionar sin mover datos a
↳través de la red)
# para que no nos queden 25 fragmentitos diminutos de parquet en HDFS Oro y
↳solo tengamos 1 bloque.
df_oro_final = df_oro.coalesce(1)

# 4. Escribimos la maravilla a la tabla de reportes Oro
ruta_oro = "hdfs://namenode:9000/data/gold/accidents_kpi/"
df_oro_final.write.mode("overwrite").parquet(ruta_oro)
df_oro_final.show(10)
```

```
+-----+-----+-----+
|State|Severity| count|
+-----+-----+-----+
|  MI|        2|113020|
|  RI|        3|  7728|
|  MA|        3| 18623|
|  VA|        3| 51324|
|  NE|        2| 24047|
|  MD|        1|   517|
|  NE|        1|    93|
|  SC|        1|  6175|
|  KY|        2| 18255|
|  WI|        4|  3462|
+-----+-----+-----+
only showing top 10 rows
```

```
[5]: spark.stop()
```

07_Spark_Integridad_Distribuida

March 12, 2026

1 Práctica 4: Gestión de Integridad en Arquitecturas Distribuidas

Ref. Teórica: UT3 - Punto 4 (Gestión de integridad en arquitecturas distribuidas) **Tecnología:** Apache Spark (Modo Local)

En sistemas distribuidos, factores como particiones de red, caídas de nodos o productoras desincronizadas generan riesgos de **integridad**. Como vimos en la **UT3 - 4.1**, la integridad se asegura combinando **responsabilidades técnicas** (idempotencia en Spark) y **organizativas** (contratos de datos empíricos).

En esta práctica vamos a simular fallos comunes abordados en la teoría y a resolverlos con Spark puro en la Arquitectura Medallón.

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, \
    DoubleType, TimestampType

spark = SparkSession.builder \
    .appName("Practica_Integridad_UT3") \
    .master("local[*]") \
    .getOrCreate()

# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.")
```

Versión de PySpark 3.5.0.

1.0.1 Caso 1: Duplicados por Reintentos de Red

(Ref. Teórica UT3 - 4.3 Ejemplo 1: Duplicados en compras)

Problema: La red falla a mitad del envío. El sistema origen (App de pagos) no recibe el *ACK*

(Acknowledgement) y “por si acaso” vuelve a enviar la transacción. Como resultado, la **Capa Bronce** ingesta el mismo evento `payment_id` dos veces.

```
[2]: from datetime import datetime
# Simulamos los datos que acaban de llegar a nuestra Capa Bronce (Nota el ↵
    ↪payment_id 'P-101' duplicado)
datos_bronce = [
    ("P-100", "userA", 50.0, datetime(2026, 1, 1, 10, 0)),
    ("P-101", "userB", 20.0, datetime(2026, 1, 1, 10, 5)), # ORIGINAL
    ("P-101", "userB", 20.0, datetime(2026, 1, 1, 10, 5)), # RETRY FANTASMA
    ("P-102", "userA", 15.0, datetime(2026, 1, 1, 10, 10))
]
columnas = ["payment_id", "user_id", "amount", "timestamp"]

df_crudo = spark.createDataFrame(datos_bronce, columnas)
print(" Crudo en Capa Bronce:")
df_crudo.show()

# PREGUNTA: ¿Qué pasa si el equipo de BI (Capa Oro) suma esto directamente?
print(" Ingresos FALSOS en Capa Oro sin deduplicar:")
df_crudo.agg(sum("amount")).show()
```

Crudo en Capa Bronce:

payment_id	user_id	amount	timestamp
P-100	userA	50.0	2026-01-01 10:00:00
P-101	userB	20.0	2026-01-01 10:05:00
P-101	userB	20.0	2026-01-01 10:05:00
P-102	userA	15.0	2026-01-01 10:10:00

Ingresos FALSOS en Capa Oro sin deduplicar:

sum(amount)
105.0

Solución (Capa Plata): Idempotencia con `dropDuplicates` El Data Engineer debe garantizar que el pipeline sea idempotente (procesar el mismo archivo mil veces no cambia el resultado).

```
[3]: # Construimos nuestra tabla Plata asegurando la unicidad del identificador ↵
    ↪transaccional
df_plata_idempotente = df_crudo.dropDuplicates(["payment_id"])

print(" Datos reales (Plata):")
```

```
df_plata_idempotente.show()

print(" Ingresos REALES (Oro):")
df_plata_idempotente.agg(sum("amount")).show()
```

```
Datos reales (Plata):
+-----+-----+-----+-----+
|payment_id|user_id|amount|          timestamp|
+-----+-----+-----+-----+
|      P-100|  userA|  50.0|2026-01-01 10:00:00|
|      P-101|  userB|  20.0|2026-01-01 10:05:00|
|      P-102|  userA|  15.0|2026-01-01 10:10:00|
+-----+-----+-----+-----+
```

```
Ingresos REALES (Oro):
+-----+
|sum(amount)|
+-----+
|         85.0|
+-----+
```

1.0.2 Caso 2: Cierre Diario vs Late Events

(Ref. Teórica UT3 - 4.3 Ejemplo 2: Datos tardíos rompen el cierre diario)

Problema: Tienes un Job programado que calcula “Las ventas de ayer”. Pero un nodo de una tienda se quedó sin Wi-Fi y manda a tu servidor hoy las ventas de... ayer tras horas de latencia. Si sobrescribes todo, jodes la contabilidad histórica (el Dashboard de ayer “cambia” solo).

```
[4]: from pyspark.sql.functions import to_date

# 1. Este es nuestro histórico Oro calculado ayer (3 de Enero)
kpi_historico_oro = [
    ("2026-01-02", 5000.0), # Ayer calculamos 5K
    ("2026-01-01", 3000.0)
]
df_historico_oro = spark.createDataFrame(kpi_historico_oro, ["fecha_venta",
    ↪ "total_facturado"])
print(" Histórico Oficial en Base de Datos (Oro):")
df_historico_oro.show()

# 2. Hoy (4 de Enero), nos llega un pequeño Excel rezagado (Late Event) con
    ↪ transacciones del día 2
datos_tardios = [
    ("P-999", "userC", 100.0, datetime(2026, 1, 2, 23, 50)) # Ojo, es del día 2!
```

```
]
df_tardio = spark.createDataFrame(datos_tardios, ["payment_id", "user_id", "amount", "timestamp"]) \
    .withColumn("fecha_venta_rezagada", to_date("timestamp"))
print(" Eventos Tardíos llegados a Plata hoy:")
df_tardio.show()
```

Histórico Oficial en Base de Datos (Oro):

```
+-----+-----+
|fecha_venta|total_facturado|
+-----+-----+
| 2026-01-02|          5000.0|
| 2026-01-01|          3000.0|
+-----+-----+
```

Eventos Tardíos llegados a Plata hoy:

```
+-----+-----+-----+-----+-----+
|payment_id|user_id|amount|          timestamp|fecha_venta_rezagada|
+-----+-----+-----+-----+-----+
|      P-999|  userC| 100.0|2026-01-02 23:50:00|          2026-01-02|
+-----+-----+-----+-----+-----+
```

Solución: Reconciliación o Reproceso Controlado Defines un SLA: “Los datos son definitivos pasadas 72hs”. Si llega algo en ese margen, el pipeline debe hacer un *Merge* o reprocesar explícitamente solo esa partición de fecha en la tabla `df_historico_oro`.

1.0.3 Caso 3: Cambio de Esquema Silencioso y Corrupción

(Ref. Teórica UT3 - 4.3 Ejemplo 3: Cambio de esquema silencioso)

Problema: En origen, siempre enviaban el peso del paquete como número. Una actualización en su App sin avisarnos ahora lo envía como texto ("14 Kg"). Si usamos lecturas relajadas (como CSV con `inferSchema=True`), Spark intentará parsear “14 Kg” y al fallar matemáticamente, escribirá Null sin detener el Job. **¡Basura entra, Basura sale!**

```
[5]: # 1. Archivo que nos mandó la App engañándonos
# Simulamos que esto estaba escrito en CSV:
datos_corruptos = [
    ("Package1", "14"),
    ("Package2", "15 Kg"), # AQUÍ ESTÁ EL FALLO DE ORIGEN (String en vez de
    ↪Double)
    ("Package3", "12")
]
df_bad = spark.createDataFrame(datos_corruptos, ["product", "weight_kg"]) #
    ↪Spark lo traga como String
```

```
print(" Lo que pasa si el catálogo de metadatos o el contrato no tiene Schema_
↳Fijo:")
# Intentar hacer matemáticas sobre peso de repente te dará Nulls
df_bad.select(col("product"), col("weight_kg").cast("double")).show()
```

Lo que pasa si el catálogo de metadatos o el contrato no tiene Schema Fijo:

```
+-----+-----+
| product|weight_kg|
+-----+-----+
|Package1|    14.0|
|Package2|    NULL|
|Package3|    12.0|
+-----+-----+
```

Solución (Capa Plata): Contratos de Datos (Schema Structs) Tu código en Spark debe imponer un contrato estricto (**StructType**). De esta forma, si el origen cambia el esquema, Spark lanzará una alerta en la fase de testeo de calidad (o guardará el registro en cuarentena / Dead-Letter-Queue) antes de envenenar la tabla de ventas de tu Analista de Negocio.

```
[6]: # Ejemplo de un contrato estricto, definido por el Data Engineer en Metadatos
# (Es la única forma de garantizar semántica)
contrato_estriicto = StructType([
    StructField("product", StringType(), False), # Falso = NO Nulos permitidos
    StructField("weight_kg", DoubleType(), True)
])

print(" Usar .schema(contrato_estriicto) en tu comando spark.read.csv() obliga_
↳a Spark a validar la estructura.")
```

Usar `.schema(contrato_estriicto)` en tu comando `spark.read.csv()` obliga a Spark a validar la estructura.

```
[7]: spark.stop()
```